

Search Techniques and Reasoning

Numerical Worksheet

CSE276 • Artificial Intelligence Foundations • Unit II

School of Computer Science & Engineering

Solutions copy

Carried in from Unit I you already know how to write a problem as a state space: an *initial state*, a set of *actions*, a *transition model*, a *goal test*, and a *path-cost function*. This worksheet *uses* that skill. It does not retest it. Wherever a graph appears: nodes are states, edges are actions, and the number on an edge is that action's cost. Everything in the next section is new, and all of it is Unit II.

How to use this worksheet. Each of the six parts is built the same way, in this order:

1	The concept	the idea in words, before any symbol is used.
2	The picture	the same idea drawn, so you can see what the formula does.
3	How it differs	the method it is most often confused with, set beside it.
4	Key points	the short list to revise from. If you read nothing else, read this.
5	The numericals	two fully worked calculations, small enough to check by hand.
6	Questions	long questions with full solutions, then short drills.

The worked examples use *different numbers* from the questions, so they teach the method without giving an answer away. Round decimals to four places unless a question says otherwise.

Contents

1	The Unit II formulas	2
2	Part A — Uninformed search and what it costs	5
3	Part B — Heuristic functions	12
4	Part C — Informed search: Best First, Hill Climbing and A*	20
5	Part D — Knowledge representation: semantic networks and frames	29
6	Part E — Production systems and logic	36
7	Part F — Reasoning under uncertainty with Bayes' theorem	46

1 The Unit II formulas

Every formula you need for this unit is on the next three pages. Each one is followed by a plain-English line saying what it actually means.

1. Path cost the number a search is trying to make small. Add up the cost of every action along the path.

$$g(\text{path}) = c_1 + c_2 + \dots + c_r$$

In plain words: how much the journey cost you so far.

On an **unweighted** graph every edge costs 1, so path cost is just the number of edges. On a **weighted** graph the route with fewest edges need not be the cheapest. That is the whole reason Breadth First Search is not always the right answer.

2. Size of the search tree how many nodes a blind search makes before it reaches depth d . Each level is b times the size of the one above, so the levels form a geometric series.

$$N = 1 + b + b^2 + \dots + b^d = \frac{b^{d+1} - 1}{b - 1}$$

where

b the branching factor — how many successors a node has

d the depth of the shallowest goal

m the greatest depth the search may reach

In plain words: add up the levels; the formula is the fast way to do it.

3. Time and space of the two blind searches

BFS: $O(b^d)$ time, $O(b^d)$ space DFS: $O(b^m)$ time, $O(bm)$ space

In plain words: BFS stores a whole level, so its memory is as bad as its running time. DFS stores one branch, so its memory is small. Memory is usually what stops you first.

4. The three functions of informed search

$g(n)$ = cost already paid from start to n , $h(n)$ = estimated cost from n to the goal

$$f(n) = g(n) + h(n)$$

In plain words: g is the past, h is the future, f is the estimated total.

Uniform Cost Search expands the smallest g . Best First Search expands the smallest h . A* expands the smallest f . Put $h(n) = 0$ and A* becomes Uniform Cost Search.

5. Admissibility — the condition for A* to be optimal

$$h(n) \leq h^*(n) \quad \text{for every node } n$$

where $h^*(n)$ is the *true* cheapest cost from n to a goal.

In plain words: the guess must never be too big. Being too optimistic is allowed; being pessimistic is not.

6. Consistency (also called monotonicity) — the stronger condition

$$h(n) \leq c(n, n') + h(n') \quad \text{for every node } n \text{ and every successor } n'$$

In plain words: one step of travel cannot cut the estimate by more than the step costs. Every consistent heuristic is admissible. The reverse is not true.

7. Dominance given two admissible heuristics, h_2 dominates h_1 when

$$h_2(n) \geq h_1(n) \quad \text{for every node } n$$

In plain words: the bigger admissible guess is the better one. It is closer to the truth, so it prunes more and A expands fewer nodes.*

8. The two 8-puzzle heuristics both admissible, and the standard example of dominance.

$h_1(n)$ = number of misplaced tiles (the blank is not a tile)

$$h_2(n) = \sum_{\text{tiles}} (|r - r_{\text{goal}}| + |c - c_{\text{goal}}|) \quad (\text{total Manhattan distance})$$

In plain words: h_1 counts how many tiles are wrong. h_2 counts how far each wrong tile has to travel. Both are admissible because one move fixes at most one tile, and changes one tile's distance by exactly 1.

9. Effective branching factor the fair way to compare two heuristics on real runs. If A* generated N nodes and found a solution at depth d , then b^* is the branching factor a *uniform* tree of $N + 1$ nodes and depth d would have:

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

In plain words: if the search had been a neat uniform tree, how wide would it have been? Closer to 1 is better.

10. Hill Climbing keep no frontier at all. Look at the neighbours of the current state, move to the best one, and stop when no neighbour is better.

move to n' only if $h(n') < h(n)$ (when h is being minimised)

In plain words: always walk downhill; stop where you can no longer descend. Uses $O(1)$ memory and gives no guarantee at all.

11. Inheritance in a semantic network or frame system a property stated once at a general node is inherited by everything below it. Storing p shared properties for n instances grouped under s subclasses costs

$$\text{flat store} = n \times p, \quad \text{inherited store} = p + s + n$$

In plain words: p properties written once at the top, plus one IS-A link per subclass and one per instance. The saving grows as n grows.

12. Size of a truth table for n propositional symbols,

$$\text{rows} = 2^n$$

A sentence is **valid** when it is true in every row, **satisfiable** when it is true in at least one row, and $KB \models \alpha$ (the knowledge base *entails* α) when every row that makes KB true also makes α true.

In plain words: one row per combination of true/false. Valid = true always. Satisfiable = true somewhere. Entailed = true wherever the KB is true.

13. What quantifiers replace over a finite domain of n objects, a quantified sentence is shorthand for a propositional one:

$$\forall x P(x) \equiv P(o_1) \wedge P(o_2) \wedge \dots \wedge P(o_n), \quad \exists x P(x) \equiv P(o_1) \vee P(o_2) \vee \dots \vee P(o_n)$$

With p unary and q binary predicates over n objects, a propositional encoding needs

$$\text{symbols} = pn + qn^2, \quad \text{truth-table rows} = 2^{pn+qn^2}$$

In plain words: $\forall$ is a long AND, $\exists$ is a long OR. One quantified sentence replaces a list that grows with the size of the world.

14. Bayes' theorem the rule for revising a belief after seeing evidence.

$$P(H | E) = \frac{P(E | H) P(H)}{P(E)}, \quad P(E) = P(E | H)P(H) + P(E | \neg H)P(\neg H)$$

where

$P(H)$ the *prior* — what you believed before the evidence

$P(E | H)$ the *likelihood* — how well H explains E

$P(E)$ the *evidence* — how likely E was at all

$P(H | E)$ the *posterior* — what you believe now

In plain words: top line = the cases you want. Bottom line = all the cases where the evidence shows up, from any cause. Divide.

15. The same rule in odds form usually faster, and much harder to get wrong.

$$\underbrace{\frac{P(H | E)}{P(\neg H | E)}}_{\text{posterior odds}} = \underbrace{\frac{P(H)}{P(\neg H)}}_{\text{prior odds}} \times \underbrace{\frac{P(E | H)}{P(E | \neg H)}}_{\text{likelihood ratio}} \quad \text{then} \quad P = \frac{\text{odds}}{1 + \text{odds}}$$

In plain words: multiply the odds by the likelihood ratio, then convert odds back to a probability at the very end.

16. Several pieces of evidence, assumed independent the form an expert system actually uses.

$$P(H | E_1, E_2, \dots) \propto P(H) P(E_1 | H) P(E_2 | H) \dots$$

In plain words: score each hypothesis as prior $\times$ all its likelihoods, then divide each score by the total so they add up to 1.

The worksheet runs in six parts, following the Unit II syllabus in order. Part A is uninformed search — BFS, DFS and what they cost. Part B is heuristic functions and admissibility. Part C is informed search — Best First, Hill Climbing and A. Part D is knowledge representation — semantic networks and frames. Part E is production systems and logic. Part F is reasoning under uncertainty with Bayes' theorem.*

2 Part A — Uninformed search and what it costs

1. The concept

What “uninformed” means the algorithm is given the states and the legal actions, and nothing else. It has no idea which direction the goal is in. So it cannot aim — it can only be systematic. Everything expensive about blind search follows from that one missing piece of information.

Breadth First Search keeps its frontier in a **queue** and always expands the shallowest unexpanded node. So it finishes every node at depth d before it touches any node at depth $d + 1$. It is **complete** whenever b is finite, and it returns the solution with the *fewest edges*.

Depth First Search keeps its frontier in a **stack** and always expands the deepest unexpanded node. It follows one branch to its end, then backtracks. It is **not complete**: one infinite branch traps it forever. It is also **not optimal**. What it buys with those two losses is memory. It holds one branch, not one whole level.

The condition students lose marks for BFS is optimal *only when every step costs the same*. BFS counts edges. It has no notion of cost at all. On a road network, the route through the fewest junctions is often not the shortest in kilometres — and BFS will hand it to you anyway. The fix is **Uniform Cost Search**: expand the smallest $g(n)$ instead of the shallowest node.

2. The picture

Two things are worth seeing rather than reading. First, the order in which the two searches touch the same tree — Figure 1. Second, what each one is holding in memory while it does so. That second point is where the real difference lives.

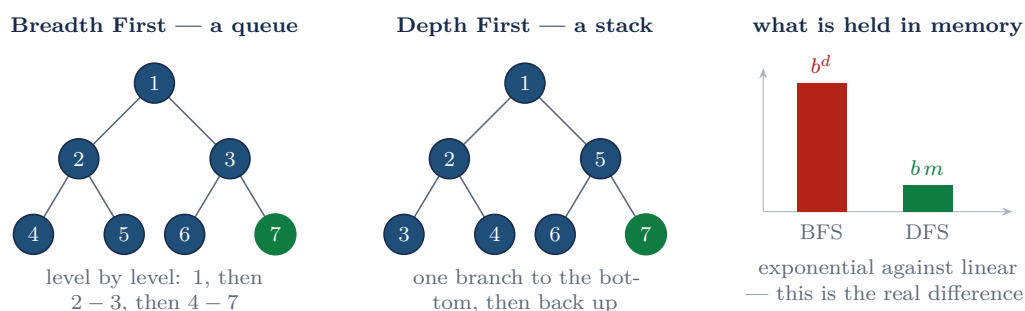


Figure 1: The same seven-node tree, two strategies. The numbers are the *order of expansion*; the green node is the goal. Both searches find it here — but they hold completely different amounts in memory while doing so.

3. How it differs — Breadth First against Depth First

One line of code separates the two: whether you take the next node from the *front* of the frontier or from the *back*. Everything in this table follows from that one line.

	Breadth First Search	Depth First Search
Frontier	a queue — first in, first out	a stack — last in, first out
Explores	every node at depth d before any at depth $d + 1$	one branch as deep as it goes, then backtracks
Complete?	yes , when b is finite	no — an infinite branch traps it
Optimal?	yes, <i>but only if every step costs the same</i>	no
Time	$O(b^d)$	$O(b^m)$
Space	$O(b^d)$ — the whole frontier	$O(bm)$ — one branch only
Use it when	the shallowest solution is wanted and memory allows	memory is tight and any solution will do

The row that matters most is the last one on space. Time you can wait for. Memory you cannot borrow. A search that needs 1.1 GB of frontier on a machine with 1 GB does not run slowly. It does not run.

4. Key points

Key points — remember these

- BFS uses a **queue**; DFS uses a **stack**. Everything else follows from that.
 - BFS: time $O(b^d)$, space $O(b^d)$. DFS: time $O(b^m)$, space $O(bm)$.
 - Total nodes down to depth d : $N = \frac{b^{d+1} - 1}{b - 1}$.
 - BFS is complete and optimal — **optimal only if all step costs are equal**. Always write that clause; marks are lost for omitting it.
 - DFS is neither complete nor optimal, but its memory is *linear*.
 - A visited (explored) set makes DFS complete on a finite graph, at a cost of $O(V)$ memory.
 - Iterative deepening = DFS memory with BFS guarantees, for roughly $(\frac{b}{b-1})$ times the nodes.
 - If step costs differ, use **Uniform Cost Search** (smallest g), not BFS.
-

5. The numericals

Worked example 2.1 — costing a blind search before you run it

A problem has branching factor $b = 3$. The shallowest goal is at depth $d = 4$. Depth First Search may descend to $m = 6$ before it finds anything.

Step 1 — count the nodes BFS generates, level by level

Each level is three times the one above it:

$$\underbrace{1}_{\text{depth 0}} + \underbrace{3}_1 + \underbrace{9}_2 + \underbrace{27}_3 + \underbrace{81}_4 = 121$$

The closed form gives the same number, and is faster when d is large:

$$N = \frac{b^{d+1} - 1}{b - 1} = \frac{3^5 - 1}{3 - 1} = \frac{243 - 1}{2} = \mathbf{121}$$

Step 2 — the space BFS needs

BFS holds the frontier, which is the widest level: $b^d = 3^4 = \mathbf{81}$ nodes. In practice it also keeps the explored set, so all 121.

Step 3 — the same two figures for DFS

$$\text{time} = O(b^m) = 3^6 = \mathbf{729} \text{ node units}, \quad \text{space} = O(bm) = 3 \times 6 = \mathbf{18} \text{ nodes}$$

Step 4 — put the two side by side

DFS does about $729/121 = 6.03$ times *more* work, and holds about $121/18 = 6.72$ times *less* of it at once.

What it tells you

This is the whole trade, in two numbers. If you have the memory and want the fewest-move answer, run BFS. If memory is the binding constraint and any solution will do, run DFS and accept that it may wander deep.

One more thing to notice: DFS's 729 is a *worst case*. It could stumble onto the goal almost immediately. BFS's 121 is what it will do every single time.

Worked example 2.2 — reading the expansion order off a tree

Here is a small tree. S is the root; G is the goal. Successors are always generated left to right.

$S \rightarrow A, B \quad A \rightarrow C, D \quad B \rightarrow E, G \quad C \rightarrow F \quad D, E, F \text{ have no successors}$

Step 1 — BFS, using a queue

Take from the front, add to the back. Bold marks the node just expanded.

step	expand	queue afterwards
1	S	A, B
2	A	B, C, D
3	B	C, D, E, G
4	C	D, E, G, F
5	D	E, G, F
6	E	G, F
7	G	goal — stop

BFS order = S, A, B, C, D, E, G path = S–B–G, 2 edges

Step 2 — DFS, using a stack

Take from the top, push successors so the leftmost comes off first.

step	expand	stack afterwards (top first)
1	S	A, B
2	A	C, D, B
3	C	F, D, B
4	F	D, B
5	D	B
6	B	E, G
7	E	G
8	G	goal — stop

DFS order = S, A, C, F, D, B, E, G

Step 3 — compare the two

	BFS	DFS
nodes expanded before the goal	6	7
largest frontier held	4	3
path returned	S-B-G	S-B-G

What it tells you

Both found the same path here, because the tree is tiny and G has only one parent. What differs is the route taken to it. DFS went all the way down the A branch — C, then F — before it ever looked at B, where the goal was sitting the whole time.

Now imagine the A branch is a thousand nodes deep. BFS still reaches G at step 7. DFS visits a thousand nodes first. That is the failure mode, and it is why DFS is called incomplete: make the branch infinite and DFS never comes back at all.

Questions

Solutions follow each question, with full working.

A1 Costing the two searches A search problem has $b = 4$, a shallowest goal at $d = 3$, and Depth First Search may reach $m = 5$.

(a) Give BFS time and space, and DFS time and space, as node counts. (b) Compute the exact number of nodes BFS generates down to depth 3, both by adding the levels and by the closed formula. (c) Which algorithm would you run if the solution must use the fewest actions, and why is that not simply a matter of taste?

Solution.

(a) The four figures

$$\begin{aligned} \text{BFS time} &= b^d = 4^3 = \mathbf{64}, & \text{BFS space} &= b^d = \mathbf{64} \\ \text{DFS time} &= b^m = 4^5 = \mathbf{1024}, & \text{DFS space} &= b m = 4 \times 5 = \mathbf{20} \end{aligned}$$

(b) The exact count, by levels

$$1 + 4 + 16 + 64 = \mathbf{85}$$

And by the formula

$$N = \frac{4^4 - 1}{4 - 1} = \frac{256 - 1}{3} = \frac{255}{3} = \mathbf{85} \quad \checkmark$$

The two agree. That is the point of doing both once: from here on, use the formula.

(c) Which to run

BFS, and it is not taste — it is a guarantee. BFS returns the solution with the fewest edges *by construction*, because it finishes each level before starting the next. DFS has no such property. The first goal it trips over may sit at depth 5 when one exists at depth 3. The cost of that guarantee is the 64 against 20 in space. If the problem were large enough that b^d did not fit in memory, the guarantee would be unavailable at any price.

A2 When memory, not time, is what stops you A search has $b = 10$ and the goal lies at $d = 6$. The machine generates 10^6 nodes per second and each node occupies 1000 bytes.

(a) How many nodes does BFS generate? How long does that take, and how much memory does it need? (b) Repeat for a goal at $d = 8$. (c) With exactly 1 GB (10^9 bytes) available, what is the deepest d that BFS can reach at all?

Solution.

(a) $d = 6$

$$N = \frac{10^7 - 1}{9} = \frac{9\,999\,999}{9} = \mathbf{1\,111\,111} \text{ nodes}$$

$$\text{time} = \frac{1\,111\,111}{10^6} = \mathbf{1.1111} \text{ seconds,} \quad \text{memory} = 1\,111\,111 \times 1000 \approx \mathbf{1.111} \text{ GB}$$

(b) $d = 8$

$$N = \frac{10^9 - 1}{9} = \mathbf{111\,111\,111} \text{ nodes}$$

$$\text{time} = \frac{111\,111\,111}{10^6} = \mathbf{111.11} \text{ seconds} \approx 1.85 \text{ minutes,} \quad \text{memory} \approx \mathbf{111.1} \text{ GB}$$

(c) *The depth that 1 GB buys*

At 1000 bytes a node, 10^9 bytes holds 10^6 nodes. So we need

$$\frac{10^{d+1} - 1}{9} \leq 10^6 \implies 10^{d+1} \leq 9 \times 10^6 + 1$$

$10^7 = 10\,000\,000$ is too big and $10^6 = 1\,000\,000$ fits, so $d + 1 = 6$ and $d = \mathbf{5}$. Check: at $d = 5$, $N = 111\,111$ nodes = 0.111 GB, comfortably inside. At $d = 6$ it is 1.111 GB, over budget.

What it tells you

Read part (b) again. Two extra levels of depth cost 100 times the time — but you would never find that out, because the machine runs out of memory at 1.111 GB long before the 111 seconds elapse. **Space is the binding constraint, not time.** That is the single most useful thing to remember about BFS. Deepening the goal by one level multiplies both by b , and no amount of tuning repairs an exponential.

A3 Fewest edges is not cheapest Three routes run from S to G:

$$\text{S-A-G} : 5 + 5, \quad \text{S-B-C-G} : 2 + 2 + 2, \quad \text{S-D-G} : 4 + 7$$

(a) Give the cost and the edge count of each route. (b) Which route does BFS return, and which is genuinely optimal? (c) The S-A edge is renegotiated to cost c . For what values of c would BFS's answer also be the optimal one?

Solution.

(a) *Cost and edges*

route	cost	edges
S-A-G	10	2
S-B-C-G	6	3
S-D-G	11	2

(b) *What each search returns*

BFS finds the shallowest goal, so it returns a **two**-edge route — S–A–G at cost 10, or S–D–G at 11, depending on which successor it happens to generate first. BFS cannot tell them apart, because it does not look at cost at all.

The optimal route is **S–B–C–G at cost 6**. BFS will not return it, because it has the *most* edges. Uniform Cost Search, which expands the smallest $g(n)$, does return it.

(c) *Making BFS accidentally right*

The S–A–G route now costs $c + 5$. It is optimal when it is no worse than the 6-cost alternative:

$$c + 5 \leq 6 \implies c \leq 1$$

What it tells you

Part (c) states the situation honestly: BFS is not optimal here, and becomes so only by luck — when the cheap route happens also to be the short one. Say the claim precisely — “BFS is optimal when all step costs are equal” — and the clause is the answer.

A4 Where DFS breaks, and what depth limits cost (a) A graph contains the cycle $S \rightarrow A \rightarrow B \rightarrow S$, and the goal G is reachable only through $S \rightarrow C \rightarrow G$. Explain what plain tree-search DFS does, and what changes if the algorithm keeps a set of already-visited states. (b) With that visited set, and a graph of V states, what is the worst number of states DFS can expand? (c) Depth First Search is now given a *depth limit*, and run repeatedly with limits 0, 1, 2, 3, 4 until it succeeds. With $b = 3$ and a goal at $d = 4$, how many nodes does that generate in total? Compare with the 121 that plain BFS generates, and with BFS’s memory.

Solution.

(a) *The cycle*

Tree-search DFS treats S reached again through A and B as a *new* node. It pushes its successors and goes round the loop $S, A, B, S, A, B, \dots$ forever. It never reaches C , so it never finds G . The search is **incomplete**, and this is what that word means in practice.

With a visited set, the second arrival at S is recognised and discarded. DFS backtracks, takes $S \rightarrow C$, and finds G . Completeness on a *finite* graph is bought with exactly one data structure.

(b) *The worst case*

Every state can be expanded at most once, so at most **V** expansions. That is the trade the visited set makes: memory proportional to the number of states, in exchange for termination.

(c) *Repeated depth-limited runs*

Each run regenerates everything above its limit, so add the runs up:

limit	nodes generated	total
0	1	1
1	1 + 3	4
2	1 + 3 + 9	13
3	1 + 3 + 9 + 27	40
4	1 + 3 + 9 + 27 + 81	121

$$\text{total} = 1 + 4 + 13 + 40 + 121 = \mathbf{179} \text{ nodes}$$

The comparison

$$\frac{179}{121} = 1.4793 \quad \longrightarrow \quad \text{about } \mathbf{48\%} \text{ more nodes generated than BFS}$$

memory: $O(bd) = 3 \times 4 = \mathbf{12}$ nodes, against BFS's $b^d = \mathbf{81}$

What it tells you

Repeating the shallow levels sounds ruinous, and it is not. The bottom level dominates the sum: with $b = 3$, the last level alone is 81 of the 121, so the repetition adds only about half again.

What you buy for that 48% is a drop from 81 nodes of memory to 12 — BFS's completeness and shortest-path guarantee at DFS's linear memory. When b is larger the overhead shrinks further, because the bottom level dominates even more heavily.

More practice Answers are given in brackets.

- i. $b = 5$, goal at $d = 3$. How many nodes does BFS generate in total? [$\frac{5^4-1}{4} = 156$]
- ii. Same problem. Give BFS space and DFS space if $m = 7$. [**BFS** $5^3 = 125$; **DFS** $5 \times 7 = 35$]
- iii. A tree has $b = 2$ and a goal at depth 10. Iterative deepening is run with limits 0 to 10. Roughly how many times more nodes does it generate than a single BFS? [**about** $\frac{b}{b-1} = 2$ **times**]
- iv. Two routes run S to G: S–X–G costing $8 + 1$, and S–Y–Z–G costing $2 + 2 + 2$. Which does BFS return, and which is optimal? [**BFS returns S–X–G (2 edges, cost 9); optimal is S–Y–Z–G at cost 6**]
- v. Why does a visited set not make DFS optimal, only complete? [**it stops repeats, but DFS still returns the first goal it meets, which need not be the shallowest or cheapest**]

3 Part B — Heuristic functions

1. The concept

What a heuristic is a function $h(n)$ that guesses how far node n still is from a goal. It is not part of the problem. It is knowledge *you* supply about the problem, and it is what turns a blind search into an informed one. Straight-line distance is the classic example: it ignores the roads, so it is only a guess, but it is a guess that always points roughly the right way.

Admissible $h(n) \leq h^*(n)$ for every node. The estimate never exceeds the true remaining cost. An admissible heuristic is allowed to be *optimistic* and forbidden to be pessimistic.

That asymmetry is not arbitrary. An optimistic estimate can only make A* look at a path it did not need to. A pessimistic one can make A* discard the best path unseen — and once discarded, it is not revisited.

Consistent $h(n) \leq c(n, n') + h(n')$ along every edge. This is a triangle inequality: the estimate cannot drop faster than you can actually travel. Consistency implies admissibility. In practice almost every heuristic anyone writes down is consistent, but the two words are not interchangeable, and the implication runs one way only.

Dominance between two *admissible* heuristics, the larger one is better. If $h_2(n) \geq h_1(n)$ everywhere, then h_2 is closer to the truth, prunes more nodes, and A* using it expands a subset of what it expands with h_1 .

Note the qualifier. Making a heuristic larger is an improvement only while it stays admissible. Push it past h^* and you have not got a better heuristic; you have got a broken one.

2. The picture

The 8-puzzle is where these definitions become concrete. Figure 2 shows one state, the goal, and the two standard heuristics measured on it — together with the corridor that admissibility confines them to.

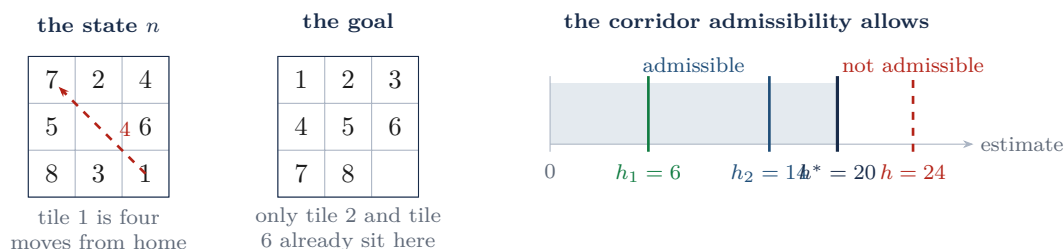


Figure 2: Both heuristics sit inside the shaded corridor, so both are admissible. h_2 sits *closer to the wall* at h^* , which is exactly what dominance means and exactly why it prunes more. Anything to the right of h^* is an overestimate, and costs A* its optimality guarantee.

3. How it differs — misplaced tiles against Manhattan distance

Both count something about how wrong the board is. They differ in *how much credit a tile gets for being wrong*, and that is the whole story.

	h_1 — misplaced tiles	h_2 — Manhattan distance
What it counts	how many tiles are not home	how far each tile is from home, added up
A tile one square out contributes	1	1
A tile four squares out contributes	1	4
Admissible?	yes — one move fixes at most one tile	yes — one move changes one tile's distance by exactly 1
Relationship	$h_1(n) \leq h_2(n)$ always	h_2 dominates h_1
Effect on A^*	explores more nodes	explores a subset of them — strictly less work

Read the third row again. h_1 throws away the difference between a tile that is nearly home and one in the far corner. It is not that h_1 is wrong — it is admissible, and A^* stays optimal with it. It is that h_1 is *needlessly* optimistic, and every unit of optimism you discard is a node A^* does not have to expand.

4. Key points

Key points — remember these

- **Admissible** means $h(n) \leq h^*(n)$: never overestimate. This is what makes A^* optimal.
 - **Consistent** means $h(n) \leq c(n, n') + h(n')$ on every edge. Consistent $\Rightarrow$ admissible. Admissible $\nRightarrow$ consistent.
 - Admissibility is checked at *one node* against h^* . Consistency is checked across *one edge*. They are different tests on different data.
 - One failing node, or one failing edge, is enough to break the property. There is no partial credit.
 - Between admissible heuristics, **bigger is better** — that is dominance. $h_2 \geq h_1$ everywhere means A^* with h_2 expands a subset of A^* with h_1 .
 - $h = 0$ is admissible (A^* becomes Uniform Cost Search). $h = h^*$ is admissible and perfect — and computing it means solving the problem.
 - For the 8-puzzle: h_1 = misplaced tiles, h_2 = Manhattan distance, h_2 dominates h_1 . Never count the blank.
 - b^* from $N + 1 = 1 + b^* + \dots + (b^*)^d$ compares heuristics fairly across different instances. Closer to 1 is better.
-

5. The numericals

Worked example 3.1 — measuring both heuristics on one board

Take the state and the goal from Figure 2. Number the rows 1–3 from the top and the columns 1–3 from the left.

state	7	2	4
	5		6
	8	3	1

goal	1	2	3
	4	5	6
	7	8	

Step 1 — tabulate where each tile is and where it belongs

The blank is not a tile and is never counted.

tile	now	goal	misplaced?	$ \Delta r + \Delta c $
1	(3, 3)	(1, 1)	yes	$2 + 2 = 4$
2	(1, 2)	(1, 2)	no	0
3	(3, 2)	(1, 3)	yes	$2 + 1 = 3$
4	(1, 3)	(2, 1)	yes	$1 + 2 = 3$
5	(2, 1)	(2, 2)	yes	$0 + 1 = 1$
6	(2, 3)	(2, 3)	no	0
7	(1, 1)	(3, 1)	yes	$2 + 0 = 2$
8	(3, 1)	(3, 2)	yes	$0 + 1 = 1$

Step 2 — h_1 is the count of “yes” in column four

Tiles 1, 3, 4, 5, 7, 8 are out of place, so

$$h_1(n) = \mathbf{6}$$

Step 3 — h_2 is the total of the last column

$$h_2(n) = 4 + 0 + 3 + 3 + 1 + 0 + 2 + 1 = \mathbf{14}$$

Step 4 — check the relationship

$14 \geq 6$, so h_2 dominates h_1 on this node. It does so on every node, because a misplaced tile contributes exactly 1 to h_1 and at least 1 to h_2 .

What it tells you

Two things, and the second is the one worth carrying away.

First, $h_2 = 14$ is a *lower bound you get for free*. No sequence of fewer than 14 moves can solve this board, because every move changes one tile’s Manhattan distance by exactly 1, and 14 units of distance must be removed. The true optimum for this board is 20 moves — so h_1 underestimates by 14 and h_2 by only 6.

Second, that gap is the work. A* with h_1 must expand everything with $f < 20$; because h_1 is so optimistic, an enormous number of nodes qualify. With h_2 the same test is far more selective. Same guarantee, same answer, a fraction of the nodes — purely because the estimate is closer to the truth.

Worked example 3.2 — checking admissibility and consistency on the same graph

A four-node graph, with the edges and their costs:

$$S-A = 2, \quad S-B = 5, \quad A-G = 6, \quad B-G = 2$$

and a proposed heuristic

$$h(S) = 6, \quad h(A) = 5, \quad h(B) = 2, \quad h(G) = 0.$$

Step 1 — work out the true costs h^ , starting at the goal*

Always work backwards. $h^*(G) = 0$ by definition.

$$h^*(A) = 6 \text{ (only route A-G),} \quad h^*(B) = 2 \text{ (only route B-G)}$$

$$h^*(S) = \min \left(\underbrace{2+6}_{\text{via A}}, \underbrace{5+2}_{\text{via B}} \right) = \min(8, 7) = 7$$

Step 2 — test admissibility, node by node

The test is $h(n) \leq h^*(n)$.

node	$h(n)$	$h^*(n)$	$h \leq h^*$?
S	6	7	✓
A	5	6	✓
B	2	2	✓ (equality is allowed)
G	0	0	✓

Every node passes, so h is **admissible**.

Step 3 — test consistency, edge by edge

The test is $h(n) \leq c(n, n') + h(n')$. Check each edge in the direction the search would travel.

edge	$h(n)$	$c + h(n')$	holds?
$S \rightarrow A$	6	$2 + 5 = 7$	✓
$S \rightarrow B$	6	$5 + 2 = 7$	✓
$A \rightarrow G$	5	$6 + 0 = 6$	✓
$B \rightarrow G$	2	$2 + 0 = 2$	✓

Every edge passes, so h is also **consistent**.

Step 4 — now break it, and see which test notices

Change one value: let $h(A) = 5$ become $h(A) = 6$.

Admissibility: $6 \leq h^*(A) = 6$ ✓ — still fine.

Consistency, edge $A \rightarrow G$: is $6 \leq 6 + 0 = 6$? ✓ — still fine.

Consistency, edge $S \rightarrow A$: is $6 \leq 2 + 6 = 8$? ✓ — still fine.

So that change breaks nothing. Try instead $h(B) = 2 \rightarrow h(B) = 4$.

Admissibility: $4 > h^*(B) = 2$ ✗ — **fails**.

Consistency, edge $B \rightarrow G$: is $4 \leq 2 + 0 = 2$? ✗ — **fails too**.

What it tells you

The two tests are asking different questions but they are not independent. Consistency is the stronger condition, so breaking it is easier: any heuristic that fails admissibility somewhere will also fail consistency somewhere, but the reverse does not hold.

The practical routine for an exam is fixed and worth memorising:

- (1) compute h^* backwards from the goal;
- (2) check $h \leq h^*$ at every node — that is admissibility;
- (3) check $h(n) \leq c + h(n')$ on every edge — that is consistency;
- (4) one failure anywhere is enough to say “no”.

Questions

B1 Both heuristics on two boards The goal is the standard one used throughout this part. Compute h_1 and h_2 for each of these states.

$$P = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline & 7 & 8 \\ \hline \end{array} \quad Q = \begin{array}{|c|c|c|} \hline 8 & 1 & 3 \\ \hline 4 & & 2 \\ \hline 7 & 6 & 5 \\ \hline \end{array}$$

(a) Give h_1 and h_2 for P , and state the true optimal solution length. (b) Give h_1 and h_2 for Q . The true optimum for Q is 14 moves; say by how much each heuristic underestimates. (c) P makes the two heuristics agree and Q does not. What feature of a board makes them agree, and why does that mean h_1 is never *more* informative than h_2 ?

Solution.

(a) *Board P*

Only tiles 7 and 8 are disturbed:

tile	now	goal	$ \Delta r + \Delta c $
7	(3, 2)	(3, 1)	1
8	(3, 3)	(3, 2)	1

$$h_1(P) = 2, \quad h_2(P) = 1 + 1 = 2$$

The true optimum is also **2**: slide 7 left into the blank, then slide 8 left. Both heuristics are exact here.

(b) *Board Q*

tile	now	goal	misplaced?	$ \Delta r + \Delta c $
1	(1, 2)	(1, 1)	yes	$0 + 1 = 1$
2	(2, 3)	(1, 2)	yes	$1 + 1 = 2$
3	(1, 3)	(1, 3)	no	0
4	(2, 1)	(2, 1)	no	0
5	(3, 3)	(2, 2)	yes	$1 + 1 = 2$
6	(3, 2)	(2, 3)	yes	$1 + 1 = 2$
7	(3, 1)	(3, 1)	no	0
8	(1, 1)	(3, 2)	yes	$2 + 1 = 3$

$$h_1(Q) = 5 \quad (\text{tiles } 1, 2, 5, 6, 8), \quad h_2(Q) = 1 + 2 + 0 + 0 + 2 + 2 + 0 + 3 = 10$$

Against the true optimum of 14:

$$h_1 \text{ underestimates by } 14 - 5 = 9, \quad h_2 \text{ underestimates by } 14 - 10 = 4$$

h_2 is more than twice as close, which is dominance measured rather than asserted.

(c) *When they agree*

They agree exactly when *every misplaced tile is one square from home*, because then each contributes 1 to both sums. That is what happens on P .

The general statement is stronger than “they sometimes agree”. A misplaced tile contributes exactly 1 to h_1 and *at least* 1 to h_2 ; a correctly placed tile contributes 0 to both. Summing over the tiles gives $h_2(n) \geq h_1(n)$ at **every** node without exception.

So h_1 can equal h_2 but can never exceed it. There is no board on which h_1 is the more informative of the two. That is what “ h_2 dominates h_1 ” asserts, and it is why the choice between them is not a judgement call.

B2 Testing admissibility Four nodes have known true remaining costs, and two heuristics are proposed:

node	n_1	n_2	n_3	n_4
$h^*(n)$	10	7	4	12
$h_A(n)$	8	7	3	11
$h_B(n)$	9	8	3	10

(a) Which of h_A , h_B is admissible? Name every node that fails. (b) State the consequence of running A* with the failing heuristic. (c) $h_A(n_2) = 7$ equals $h^*(n_2)$ exactly. Is that permitted, and what would a heuristic that equalled h^* everywhere do to the search?

Solution.

(a) Check $h \leq h^*$ node by node

	n_1	n_2	n_3	n_4	verdict
h_A	$8 \leq 10$ ✓	$7 \leq 7$ ✓	$3 \leq 4$ ✓	$11 \leq 12$ ✓	admissible
h_B	$9 \leq 10$ ✓	$8 > 7$ ✗	$3 \leq 4$ ✓	$10 \leq 12$ ✓	not admissible

h_B fails at n_2 alone — and one node is enough. Admissibility is a statement about *every* node; there is no partial credit.

(b) *The consequence*

A* with h_B still terminates and still returns a path, but the path may be **sub-optimal**.

The mechanism is worth stating precisely. n_2 is made to look 1 unit more expensive than it is. So if the genuinely cheapest route runs through n_2 , A* may expand a goal on a worse route first and stop — with n_2 still sitting unexpanded on the frontier. A* stops at the first goal it *expands*, and it never comes back.

Note what is *not* lost: completeness. An overestimate costs the optimality guarantee, not termination.

(c) *Equality, and the perfect heuristic*

Yes, $h = h^*$ is permitted. Admissibility is $\leq$, not $<$.

A heuristic equal to h^* everywhere is the best possible one: $f(n) = g(n) + h^*(n)$ is then the true cost of the best path through n , which is constant along the optimal path and strictly larger off it. A* would walk straight to the goal, expanding *only* the optimal path.

That is the ideal, and it is also the joke. Computing h^* means solving the problem, which is what you asked A* to do. Every real heuristic is a cheap approximation to a quantity exactly as hard as the original task, and dominance measures how close to it you got.

B3 Testing consistency Two edges are inspected in a graph whose heuristic values are known:

edge 1: $n \rightarrow n'$ with $c = 3$, $h(n) = 9$, $h(n') = 5$

edge 2: $m \rightarrow m'$ with $c = 4$, $h(m) = 6$, $h(m') = 5$

(a) Apply the consistency test to each edge. (b) Edge 1 fails. Does that prove h is inadmissible? (c) State the relationship between the two properties in one sentence, and say which direction the implication runs.

Solution.

(a) *The test is $h(n) \leq c(n, n') + h(n')$*

edge 1: $9 \leq 3 + 5 = 8 ? \quad 9 > 8 \Rightarrow \text{fails}$

edge 2: $6 \leq 4 + 5 = 9 ? \quad 6 \leq 9 \Rightarrow \text{holds}$

So h is **not consistent**. Consistency must hold on every edge, and edge 1 breaks it.

(b) *Does that make it inadmissible?*

No. The two properties are different tests. Consistency compares h at *two neighbouring nodes*. Admissibility compares h at *one node* against the true remaining cost h^* at that node. Nothing in the data above mentions h^* , so nothing above can settle admissibility either way.

Concretely: if the true costs happened to be $h^*(n) = 11$ and $h^*(n') = 6$, then $9 \leq 11$ and $5 \leq 6$, and h is perfectly admissible — while still failing the edge-1 triangle inequality.

(c) *The relationship*

Every consistent heuristic is admissible, but an admissible heuristic need not be consistent. The implication runs from consistency to admissibility and not back.

Why it matters in practice: with a consistent heuristic, f never decreases along a path, so the first time A^* expands a node it already has that node's cheapest g , and A^* need never reopen a closed node. With a merely admissible one it can — and an implementation that forgets to allow reopening will quietly return a sub-optimal answer.

B4 Effective branching factor On two problem instances, two heuristics are run and the nodes generated counted:

h_X : $N = 6$ nodes at depth $d = 2$, h_Y : $N = 39$ nodes at depth $d = 3$

(a) Compute b^* for each. (b) These were different instances, so the raw node counts 6 and 39 cannot be compared directly. Explain what b^* does about that. (c) A third heuristic reports $b^* = 1.02$. What does that say about how the search behaved?

Solution.

(a) *Solve $N + 1 = 1 + b^* + \dots + (b^*)^d$*

For h_X , with $N = 6$ and $d = 2$:

$$1 + b^* + (b^*)^2 = 7 \implies (b^*)^2 + b^* - 6 = 0 \implies (b^* + 3)(b^* - 2) = 0$$

Discard the negative root: $b^* = 2$.

For h_Y , with $N = 39$ and $d = 3$:

$$1 + b^* + (b^*)^2 + (b^*)^3 = 40$$

Try $b^* = 3$: $1 + 3 + 9 + 27 = 40 \checkmark$. So $b^* = 3$.

(b) *What b^* normalises away*

Depth. A search that generates 39 nodes to reach depth 3 is not obviously worse than one generating 6 to reach depth 2 — the second had a much easier instance.

b^* answers the depth-independent question: *if this search had been a uniform tree, how wide would it have been?* Two and three are directly comparable in a way that six and thirty-nine are not, so h_X is the better heuristic here.

It is also the figure that survives a change of hardware, which raw node counts and wall-clock times do not.

(c) *Reading* $b^* = 1.02$

$b^* = 1$ would mean the search expanded a single path with no branching at all — it walked straight to the goal. At 1.02 the search is very close to that: the heuristic is near-perfect on this instance, pruning almost every alternative before it is generated.

The practical reading: node count grows as $(b^*)^d$. At $b^* = 1.02$ the search stays feasible to depths where a b^* of 3 would have exhausted memory many levels earlier. Small differences in b^* are exponentially large differences in reach.

More practice Answers are given in brackets.

- i. A tile sits at $(1, 1)$ and belongs at $(3, 2)$. What does it contribute to h_1 , and to h_2 ? [**1 and $|1 - 3| + |1 - 2| = 3$**]
- ii. $h^*(n) = 9$. Which of $h = 0$, $h = 9$, $h = 10$ are admissible? [**0 and 9; 10 is not**]
- iii. An edge $n \rightarrow n'$ costs 2, with $h(n) = 8$ and $h(n') = 5$. Is it consistent? [**$8 \leq 2 + 5 = 7$ is false — not consistent**]
- iv. h_1 and h_2 are both admissible. Is $\max(h_1, h_2)$ admissible? Does it dominate both? [**yes and yes — this is the standard way to combine heuristics**]
- v. A^* generates $N = 12$ nodes at depth $d = 2$. Find b^* . [**$1 + b + b^2 = 13 \Rightarrow b^2 + b - 12 = 0 \Rightarrow b^* = 3$**]

4 Part C — Informed search: Best First, Hill Climbing and A*

1. The concept

All three use a heuristic and they differ only in what *else* they are willing to look at.

Best First Search, or greedy search expands whichever frontier node *looks* closest: $f(n) = h(n)$. It is fast, and it is frequently right, and it is not optimal — because it ignores the cost already paid. A node one step from the start and a node fifty steps in are treated identically if their estimates agree.

Hill Climbing keeps no frontier at all. It looks at the current state's neighbours, moves to the best one, and repeats until no neighbour is better. Its memory is $O(1)$: it remembers one state. It offers no guarantee of any kind. It halts at the first of these it meets:

- **local optimum** — every neighbour is worse, but you are nowhere near the goal;
- **plateau** — every neighbour is equal, so there is no signal to follow;
- **ridge** — every single-step move goes down, although progress is possible diagonally.

The two standard repairs are *random restarts* and *simulated annealing*, which sometimes accepts a worse move early on so the search can climb back out.

A* Search expands the smallest $f(n) = g(n) + h(n)$: cost paid plus cost estimated. It is complete, and it is optimal whenever h is admissible. It is also *optimally efficient* — no algorithm using the same heuristic expands fewer nodes. What it pays for that is memory, because it stores the frontier exactly as BFS does.

The trade, in one line A* pays memory for a guarantee. Hill Climbing pays the guarantee for memory. Best First sits in between. Neither end is free.

2. The picture

Figure 3 is the graph the first worked example runs on, with every g , h and edge cost marked, so the trace can be checked against it line by line. Figure 4 is the landscape Hill Climbing gets stuck in.

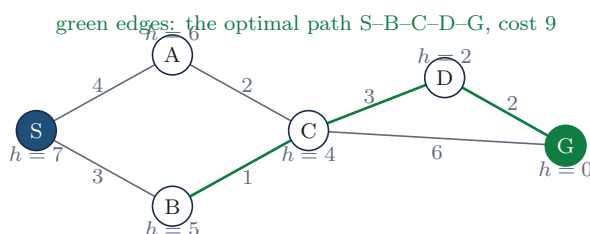


Figure 3: The search graph for the first worked example of this part. Numbers on the edges are step costs; h is the heuristic estimate of the remaining cost to G. Every h here is admissible — check any one of them against the cheapest route from that node and it comes out no larger.

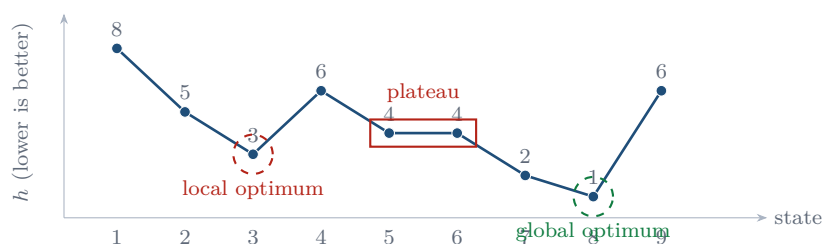


Figure 4: Hill Climbing on a one-dimensional landscape, minimising h ; the neighbours of a state are the two beside it. Starting at state 1 the search slides into state 3 and stops, two units short of the answer. It has no way of knowing that, because from where it stands every direction is uphill.

3. How it differs — the four strategies side by side

	Expands	Optimal?	Space	The characteristic failure
Uniform Cost	smallest g	yes	$O(b^d)$	explores in every direction equally; slow
Best First	smallest h	no	$O(b^d)$	follows a tempting estimate down an expensive path
A*	smallest $g + h$	yes, if h admissible	$O(b^d)$	runs out of memory
Hill Climbing	best neighbour, then forgets the rest	no	$O(1)$	stops dead at a local optimum, plateau or ridge

The first two rows are the two halves A* puts back together. Uniform Cost is A* with $h = 0$: all hindsight, no foresight. Best First is A* with g dropped: all foresight, no memory of what the journey has already cost. Each is defensible on its own, and each has a failure the other would have caught. That is precisely why $f = g + h$ is the formula that survives.

4. Key points

Key points — remember these

- Uniform Cost expands smallest g . Best First expands smallest h . A* expands smallest $f = g + h$.
- A* is complete and **optimal if h is admissible**. State the condition every time.
- A* stops when a goal is **expanded**, not when it is generated. Generating a goal early proves nothing.
- A* expands every node with $f < C^*$ and no node with $f > C^*$, where C^* is the optimal cost. That single line answers most “which nodes were never expanded?” questions.
- If a cheaper route to a frontier node is found, **update** its g and f — do not add a duplicate.
- Best First is not optimal even with an admissible heuristic. It has no g term, so it cannot correct a bad estimate.

- Hill Climbing: $O(1)$ memory, no guarantee. It fails at a **local optimum**, a **plateau** or a **ridge**.
- Random restarts with success probability p need $1/p$ attempts on average.
- An inadmissible (overestimating) h costs A^* its *optimality*, not its completeness.

5. The numericals

Worked example 4.1 — tracing A^* to the goal

Run A^* on the graph of Figure 3, from S to G. Keep a frontier of (node, g , h , f) and always expand the smallest f .

Step 1 — start

Frontier: $\{S : g = 0, h = 7, f = 7\}$. Expand S.

Step 2 — expand S

$$A : g = 0 + 4 = 4, h = 6, f = \mathbf{10} \quad B : g = 0 + 3 = 3, h = 5, f = \mathbf{8}$$

Frontier: $\{A : 10, B : 8\}$. The smallest f is B.

Step 3 — expand B

$$C : g = 3 + 1 = 4, h = 4, f = \mathbf{8}$$

Frontier: $\{A : 10, C : 8\}$. Expand C.

Step 4 — expand C

$$D : g = 4 + 3 = 7, h = 2, f = \mathbf{9} \quad G : g = 4 + 6 = 10, h = 0, f = \mathbf{10}$$

Frontier: $\{A : 10, D : 9, G : 10\}$. Expand D.

Note carefully: G is on the frontier with $f = 10$, and A^* does *not* stop. A goal that has been *generated* is only a candidate. A^* stops when a goal is *expanded* — that is, when it is the smallest f on the frontier.

Step 5 — expand D

$$G : g = 7 + 2 = 9, h = 0, f = \mathbf{9}$$

This is a cheaper route to a node already on the frontier, so the old G at $f = 10$ is replaced.

Frontier: $\{A : 10, G : 9\}$.

Step 6 — expand G

G now has the smallest f , so it is expanded, it is a goal, and the search stops.

$$\text{path} = \text{S-B-C-D-G}, \quad \text{cost} = 3 + 1 + 3 + 2 = \mathbf{9}$$

Step 7 — the expansion order

S, B, C, D, G — five expansions; A was never expanded.

What it tells you

A is the whole lesson. Its $f = 10$ exceeded the optimal solution cost of 9, so A^* never touched it. That is a theorem, not luck: A^* expands every node with $f < C^*$ and no node with $f > C^*$. A better (larger, still admissible) heuristic raises more nodes above the C^* line, which is exactly the mechanism by which dominance from Part B saves work.

Notice also step 4. Had this been Best First Search, G's $h = 0$ would have made it irresistible the moment it appeared, and the answer returned would have been the 10-cost route S–B–C–G. Carrying g is what stops that.

Worked example 4.2 — Hill Climbing, and what restarts are worth

Eight states lie in a row. Each state's neighbours are the two beside it. The value being minimised is h :

state	1	2	3	4	5	6	7	8
h	6	3	5	8	4	4	1	2

Step 1 — trace from state 1

At 1 ($h = 6$): the only neighbour is 2 ($h = 3$). Better, so move.

At 2 ($h = 3$): neighbours are 1 ($h = 6$) and 3 ($h = 5$). Both worse.

Stop at state 2, $h = 3$, after 1 move. This is a **local optimum** — the global answer is at state 7 with $h = 1$.

Step 2 — trace from state 4

At 4 ($h = 8$): neighbours 3 ($h = 5$) and 5 ($h = 4$). Best is 5, so move.

At 5 ($h = 4$): neighbours 4 ($h = 8$) and 6 ($h = 4$). Neither is *better* — state 6 is *equal*.

Stop at state 5, $h = 4$, after 1 move. This is a **plateau**. Strict hill climbing halts, because the rule is “move only if $h(n') < h(n)$ ”. A variant that allows sideways moves would step to 6, then find 7 and succeed — which is exactly why sideways moves are allowed in practice, with a cap on how many in a row.

Step 3 — trace every starting state

start	trajectory	ends at
1	1 → 2	2 ($h = 3$)
2	stuck immediately	2 ($h = 3$)
3	3 → 2	2 ($h = 3$)
4	4 → 5	5 ($h = 4$) plateau
5	stuck immediately	5 ($h = 4$) plateau
6	6 → 7	7 ($h = 1$)
7	stuck immediately	7 ($h = 1$)
8	8 → 7	7 ($h = 1$)

Three of the eight starts reach the global optimum:

$$p = P(\text{success}) = \frac{3}{8} = \mathbf{0.3750}$$

Step 4 — what restarts buy

Restarts are independent trials, so the number of attempts until the first success is geometric with mean $1/p$:

$$E[\text{attempts}] = \frac{1}{p} = \frac{8}{3} = \mathbf{2.6667}$$

The probability that k restarts *all* fail is $(1 - p)^k$:

$$(0.625)^5 = \mathbf{0.0954}, \quad (0.625)^{10} = \mathbf{0.0091}$$

So ten cheap restarts leave under a 1% chance of never seeing the answer, against a 62.5% chance from a single run.

What it tells you

A single run of Hill Climbing succeeds here only 37.5% of the time, and it fails in two distinct ways — a local optimum from starts 1–3, a plateau from starts 4–5. Neither failure is detectable from inside the algorithm: at state 2, every direction looks uphill, and the algorithm has no memory of anywhere else to go.

Restarts fix this without fixing the algorithm. Each run costs almost nothing because memory is $O(1)$, and independent cheap runs compound: 2.67 runs on average, or ten runs for near-certainty. That is the standard trade in local search — do not make one search cleverer, make many searches and keep the best.

Questions

C1 Choosing the next node An A* frontier holds three nodes:

node	P	Q	R
$g(n)$	3	5	1
$h(n)$	6	2	9

(a) Compute f for each and name the node A* expands. (b) Which node would Best First Search expand, and which would Uniform Cost Search expand? (c) R has by far the smallest g . Say in one sentence why A* is not tempted by it.

Solution.

(a) $f = g + h$

$$f(P) = 3 + 6 = 9, \quad f(Q) = 5 + 2 = 7, \quad f(R) = 1 + 9 = 10$$

A* expands **Q**.

(b) *The other two strategies*

Best First uses h alone, and the smallest h is Q 's 2 — so it also expands **Q**. Uniform Cost uses g alone, and the smallest g is R 's 1 — so it expands **R**.

The three agreeing or disagreeing on one frontier proves nothing on its own. It is on the paths that follow that they part company.

(c) *Why R is not tempting*

Because cheap so far is not the same as cheap overall: R cost only 1 to reach but still estimates 9 to finish, giving the worst projected total of the three — and f is a statement about the whole journey, not the part already travelled.

C2 A full A* trace A new graph, with undirected edges and their costs:

$$S-A = 1, \quad S-B = 4, \quad A-B = 2, \quad A-C = 2, \quad C-D = 1, \quad C-G = 2, \quad D-G = 4$$

and heuristic values

$$h(S) = 4, \quad h(A) = 3, \quad h(B) = 4, \quad h(C) = 2, \quad h(D) = 3, \quad h(G) = 0.$$

(a) Trace A* from S to G, showing the frontier after every expansion. Give the path and its cost. (b) List the expansion order and name every node A* never expanded, with the reason. (c) Verify that h is admissible at A and at D .

Solution.

(a) *The trace*

Expand S ($g = 0, f = 4$):

$$A : g = 1, f = 1 + 3 = 4 \quad B : g = 4, f = 4 + 4 = 8$$

Frontier $\{A : 4, B : 8\}$.

Expand A ($f = 4$):

$$C : g = 1 + 2 = 3, f = 3 + 2 = 5$$

$$B : g = 1 + 2 = 3, f = 3 + 4 = 7 \quad (\text{cheaper than the old } g = 4, \text{ so replace it})$$

Frontier $\{C : 5, B : 7\}$.

Expand C ($f = 5$):

$$G : g = 3 + 2 = 5, f = 5 + 0 = 5 \quad D : g = 3 + 1 = 4, f = 4 + 3 = 7$$

Frontier $\{G : 5, B : 7, D : 7\}$.

Expand G ($f = 5$) — it is a goal and has the smallest f , so stop.

$$\text{path} = S-A-C-G, \quad \text{cost} = 1 + 2 + 2 = 5$$

(b) *Expansion order*

S, A, C, G — four expansions.

B and D were generated but never expanded. Both finished with $f = 7$, and $7 > C^* = 5$, so A^* reached the goal before either could rise to the front of the queue. This is the guarantee in action: nothing with $f > C^*$ is ever expanded.

(c) *Admissibility spot-check*

At A : the cheapest route to G is $A-C-G = 2 + 2 = 4$, so $h^*(A) = 4$ and $h(A) = 3 \leq 4 \checkmark$.

At D : the routes are $D-G = 4$ and $D-C-G = 1 + 2 = 3$, so $h^*(D) = 3$ and $h(D) = 3 \leq 3 \checkmark$ — tight, and still legal, since admissibility permits equality.

C3 Where Best First Search goes wrong A small graph, with an *admissible* heuristic:

$$S-A = 1, \quad A-G = 10, \quad S-B = 2, \quad B-G = 3$$

$$h(S) = 3, \quad h(A) = 2, \quad h(B) = 3, \quad h(G) = 0$$

(a) Confirm h is admissible at A and at B . (b) Trace Best First Search from S . What path and cost does it return? (c) Trace A^* on the same graph. What path and cost does it return? (d) A^* was lured to A first as well. Why did it recover when Best First did not?

Solution.

(a) *Admissibility*

$h^*(A) = 10$ and $h(A) = 2 \leq 10 \checkmark$; $h^*(B) = 3$ and $h(B) = 3 \leq 3 \checkmark$. Both fine. $h(A)$ is merely a very poor estimate, and being a poor estimate is not the same as being inadmissible.

(b) *Best First Search, $f = h$*

At S the successors are A ($h = 2$) and B ($h = 3$). The smaller estimate wins, so it moves to A ; from A the only way on is G .

$$\text{path} = \text{S-A-G}, \quad \text{cost} = 1 + 10 = \mathbf{11}$$

(c) A^* , $f = g + h$

Expand S: $A : g = 1, f = 1 + 2 = \mathbf{3}$; $B : g = 2, f = 2 + 3 = 5$. Frontier $\{A : 3, B : 5\}$.

Expand A ($f = 3$): $G : g = 1 + 10 = 11, f = 11$. Frontier $\{B : 5, G : 11\}$.

Expand B ($f = 5$): $G : g = 2 + 3 = 5, f = 5$ — cheaper, so it replaces the $f = 11$ entry. Frontier $\{G : 5\}$.

Expand G ($f = 5$), a goal, so stop.

$$\text{path} = \text{S-B-G}, \quad \text{cost} = 2 + 3 = \mathbf{5}$$

(d) Why A^* recovered

Both algorithms were drawn to A first, by the same misleading $h(A) = 2$. The difference is what happened *next*.

Best First had already discarded B . Having committed to A , it keeps no record of the alternative and simply continues.

A^* kept B on the frontier with $f = 5$. The moment expanding A revealed the real cost of that route ($f = 11$), the B route was cheaper and got expanded instead. The g term is what turned a bad estimate into a bad estimate that was *corrected* on contact with reality.

Best First's error here is 11 against 5 — more than double the optimal cost, from an admissible heuristic.

C4 Hill Climbing, local optima and restarts Seven states lie in a row; each state's neighbours are the two beside it. The value being *minimised* is

state	1	2	3	4	5	6	7
h	7	4	6	9	5	2	3

(a) Trace Hill Climbing from state 1 and from state 4. Where does each stop, and after how many moves? (b) From how many of the seven possible starting states does Hill Climbing reach the global optimum? Express it as a success probability for a random start. (c) Using random restarts, what is the expected number of restarts before success? (d) Hill Climbing uses $O(1)$ memory here. What exactly is it that it refuses to store, and how does that produce the failure in (a)?

Solution.

(a) Two traces

From state 1 ($h = 7$): the only neighbour is 2 ($h = 4$), better, so move. At 2 ($h = 4$): neighbours are 1 ($h = 7$) and 3 ($h = 6$) — both worse. *Stop at state 2, $h = 4$, after 1 move.*

From state 4 ($h = 9$): neighbours are 3 ($h = 6$) and 5 ($h = 5$); the best is 5, so move. At 5 ($h = 5$): neighbours 4 ($h = 9$) and 6 ($h = 2$); move to 6. At 6 ($h = 2$): neighbours 5 ($h = 5$) and 7 ($h = 3$) — both worse. *Stop at state 6, $h = 2$, after 2 moves — the global optimum.*

(b) *All seven starts*

start	trajectory	ends at
1	$1 \rightarrow 2$	2 ($h = 4$)
2	stuck immediately	2 ($h = 4$)
3	$3 \rightarrow 2$	2 ($h = 4$)
4	$4 \rightarrow 5 \rightarrow 6$	6 ($h = 2$)
5	$5 \rightarrow 6$	6 ($h = 2$)
6	stuck immediately	6 ($h = 2$)
7	$7 \rightarrow 6$	6 ($h = 2$)

Four of the seven succeed:

$$P(\text{success}) = \frac{4}{7} = \mathbf{0.5714}$$

(c) *Expected restarts*

Restarts are independent trials with success probability $p = 4/7$, so the expected number of attempts is

$$\frac{1}{p} = \frac{7}{4} = \mathbf{1.75} \text{ attempts}$$

Fewer than two runs, on average, to find an answer that a single run finds only 57% of the time. That is why so cheap a fix works so well: each run costs almost nothing, and independent cheap runs compound.

(d) *What it refuses to store*

The **frontier**. A* and BFS keep every generated-but-unexpanded node, so an alternative that looked worse earlier is still available later. Hill Climbing keeps one state and throws its rejected neighbours away permanently.

That is the direct cause of the failure from state 1. On reaching state 2 it has already discarded state 1, and state 3 ($h = 6$) looks worse than staying — but 3 is the doorway to the whole right-hand valley containing the answer. The information needed to escape was seen and deleted.

$O(1)$ memory and no guarantee are not two separate facts about Hill Climbing. The second is caused by the first.

C5 What an overestimate costs A graph has

$$S-A = 2, \quad A-G = 2, \quad S-B = 3, \quad B-G = 3$$

and someone supplies $h(S) = 4$, $h(A) = 5$, $h(B) = 3$, $h(G) = 0$. **(a)** Which heuristic value is not admissible, and by how much? **(b)** Trace A* and give the path it returns. **(c)** What is the genuinely optimal path, and how much worse is A*'s answer? **(d)** What range of values for $h(A)$ would have preserved the optimal answer?

Solution.

(a) *The offending value*

The true remaining costs are $h^*(A) = 2$, $h^*(B) = 3$, $h^*(S) = 4$.

$$h(A) = 5 > 2 = h^*(A) \quad \text{an overestimate of } \mathbf{3}$$

$h(B) = 3 = h^*(B)$ and $h(S) = 4 = h^*(S)$ are both fine.

(b) *The trace*

Expand S ($f = 4$): $A : g = 2, f = 2 + 5 = 7$; $B : g = 3, f = 3 + 3 = 6$. Frontier $\{A : 7, B : 6\}$.

Expand B ($f = 6$): $G : g = 3 + 3 = 6, f = 6$. Frontier $\{A : 7, G : 6\}$.

Expand G ($f = 6$) — smallest on the frontier, and a goal, so A^* stops.

returned path = S–B–G, cost = 6

(c) *The true optimum*

$$S-A-G = 2 + 2 = 4$$

A^* returned 6 where 4 was available — 50% worse. And it did so *confidently*: nothing in the output signals that a better path exists, and A sat unexpanded on the frontier the whole time with the answer behind it.

(d) *Repairing it*

A^* stops when the goal has the smallest f . To have expanded A before the $f = 6$ goal, we need

$$f(A) = 2 + h(A) < 6 \implies h(A) < 4$$

Admissibility demands the stronger $h(A) \leq 2$, which certainly satisfies it.

So $h(A) = 3$ would have happened to work *on this graph*, while remaining an overestimate. That is the point of insisting on admissibility rather than on “close enough”. Admissibility guarantees optimality on *every* graph. Anything weaker guarantees it on the graphs where you got lucky — and you have no way to tell which those are without solving the problem first.

More practice Answers are given in brackets.

- i. A frontier holds X ($g = 3, h = 5$), Y ($g = 1, h = 6$), Z ($g = 6, h = 2$). Which does A^* expand? Best First? Uniform Cost? [$f = 8, 7, 8$; **A^* expands Y ; Best First expands Z (smallest h); Uniform Cost expands Y (smallest g)]**
- ii. A^* finds an optimal cost $C^* = 12$. A node has $f = 15$. Was it expanded? [**no — A^* expands nothing with $f > C^*$**]
- iii. A^* generates the goal with $f = 9$ while a node with $f = 7$ sits on the frontier. Does A^* stop? [**no — it expands the $f = 7$ node first**]
- iv. Hill Climbing succeeds from 2 of 10 starts. Expected restarts to succeed? Probability that 6 restarts all fail? [$1/0.2 = 5$; $0.8^6 = 0.2621$]
- v. Setting $h = 0$ turns A^* into which algorithm? Is it still optimal? [**Uniform Cost Search; yes — $h = 0$ is admissible**]

5 Part D — Knowledge representation: semantic networks and frames

1. The concept

Why represent knowledge at all search finds a path, but it must explore to do it, and Part A showed what exploring costs. Knowledge is the alternative. If the system already *knows* that a bishop stays on one colour, it never searches the moves that break the rule. After the first AI winter, the field's answer was to stop searching harder and start encoding what experts already know.

Semantic networks a graph. Nodes are concepts, and labelled edges are relationships between them. The two edges that carry most of the weight are IS-A (class membership) and HAS-A (property).

Their value is **inheritance**: a property stated once at a general node is available to everything beneath it. You assert it once instead of once per instance.

Frames the same idea built from records rather than edges. One frame per concept, each holding named **slots** and their values. A slot may carry a *default* that a more specific frame overrides, and a frame links to its parent so unfilled slots fall back up the chain. This is where object-oriented programming came from — class and inheritance and all.

What inheritance buys, and what it costs it buys storage and consistency: change the property at the top and every instance sees the change. It costs *lookup*: answering a query means walking up the chain until the slot is found, so the deeper the hierarchy, the more links a single question traverses.

2. The picture

Figure 5 draws the same knowledge twice: as a network of edges, and as a stack of frames. Follow the dashed arrow to see what a query actually does.

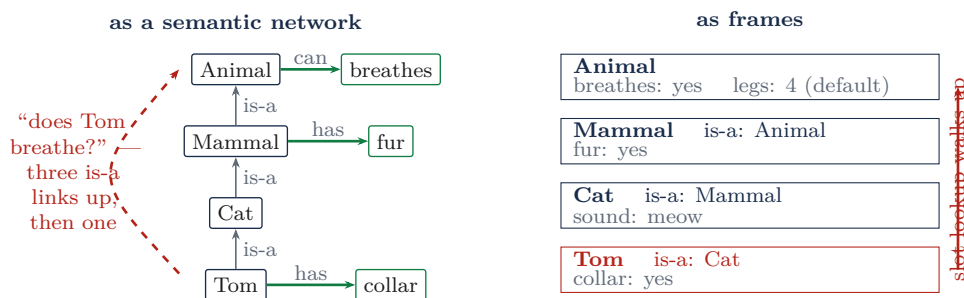


Figure 5: Six stored assertions on the left; the answer to “does Tom breathe?” is not one of them. It is *derived* by walking the IS-A chain, and that walk is what inheritance charges you for the storage it saves.

3. How it differs — semantic networks against frames

	Semantic network	Frames
Built from	nodes and labelled edges — a graph	records of named slots, one per concept
A property is	an edge to another node	a slot with a value
Inheritance	follow IS-A edges upward	follow the parent link upward
Defaults and exceptions	awkward — an edge is simply present or absent	natural — a child slot overrides the parent's default
Good for	taxonomies; anything where the <i>relationships</i> matter most	structured objects with typical values
Weak for	procedures, exceptions, quantified statements	arbitrary relations between objects

The row on defaults is the one that decides which you reach for. “Birds fly, except penguins” is a single override in a frame system, and awkward in a plain semantic network. Note also that it is not expressible in ordinary logic at all: retracting a conclusion when you learn *more* is exactly what logical entailment forbids.

4. Key points

Key points — remember these

- A semantic network is a **graph**: nodes are concepts, labelled edges are relations. IS-A gives class membership; HAS-A gives properties.
 - A frame is a **record**: named slots with values, plus a parent link. Same idea, different data structure.
 - **Inheritance** is the point of both. Say it once at the top, use it everywhere below.
 - Storage: flat = $n \times p$; inherited = $p + s + n$. The saving grows with n .
 - An **exception** costs 1 extra assertion at the node that differs, not n .
 - Slot lookup stops at the **first** frame that supplies a value — which is why an override works without any special mechanism.
 - Query cost = number of links walked. Deeper hierarchy = cheaper storage but slower lookup. That is the trade.
 - Defaults with exceptions make reasoning **non-monotonic**: learning more can withdraw a conclusion. Ordinary logic cannot do this.
-

5. The numericals

Worked example 5.1 — *what inheritance saves, counted*

A vehicle knowledge base has one general class **Vehicle** carrying $p = 6$ shared properties. Beneath it sit $s = 4$ subclasses (**Car**, **Bus**, **Truck**, **Motorcycle**), and each subclass has 3 instances, so $n = 12$ instances in all.

Step 1 — store it flat, with no inheritance

Every instance must carry every shared property itself:

$$\text{flat} = n \times p = 12 \times 6 = \mathbf{72} \text{ assertions}$$

Step 2 — store it with inheritance

The six properties are asserted once at **Vehicle**, and every subclass and every instance needs one IS-A link:

$$\text{inherited} = \underbrace{p}_6 + \underbrace{s}_4 + \underbrace{n}_{12} = \mathbf{22} \text{ assertions}$$

Step 3 — the saving

$$72 - 22 = 50 \text{ assertions saved,} \quad \frac{50}{72} = \mathbf{69.44\%}$$

Step 4 — what happens as the base grows

Push the instance count to $n = 100$, keeping $p = 6$ and $s = 4$:

$$\text{flat} = 600, \quad \text{inherited} = 6 + 4 + 100 = 110, \quad \frac{600}{110} = \mathbf{5.45\times}$$

The flat store grows as $n \times p$. The inherited store grows as n plus a constant. The gap widens without limit.

Step 5 — the exception, and the query cost

Motorcycles have two wheels, not the four the general class asserts. That is one extra assertion at the **Motorcycle** node, taking the total to 23. An exception costs 1, not n .

The bill arrives at query time. Asking “does **car_7** have brakes?” means walking **car_7** → **Car** → **Vehicle**: two IS-A links, because the hierarchy is two deep.

What it tells you

Inheritance is a trade, not a free win. You have converted *stored* facts into *derived* ones, paying a graph walk for each query in exchange for storing 22 things instead of 72. It pays off overwhelmingly when the hierarchy is shallow and wide, which is what real taxonomies look like.

It also buys something the count does not show: consistency. In the flat store, correcting one shared property means finding and editing 12 separate assertions — and missing one leaves the base quietly self-contradictory.

Worked example 5.2 — resolving slots through a frame hierarchy

Four frames, each one more specific than the last:

Vehicle	wheels: 4 (<i>default</i>), engine: yes, legal-on-road: yes
Bicycle	is-a: Vehicle, wheels: 2 , engine: no
Racer	is-a: Bicycle, gears: 22
bike_9	is-a: Racer, colour: red

Step 1 — the lookup rule

To find a slot on **bike_9**: look on **bike_9** itself; if it is not there, follow **is-a** to the parent and look again; repeat until a value is found or the chain runs out. **Stop at the first value found.**

Step 2 — resolve five slots

slot	value	links walked	found at
colour	red	0	bike_9
gears	22	1	Racer
wheels	2	2	Bicycle — — — <i>theoverride</i>
engine	no	2	Bicycle — — — <i>theoverride</i>
legal-on-road	yes	3	Vehicle

$$\text{total links walked} = 0 + 1 + 2 + 2 + 3 = 8$$

Step 3 — why the override needs no special machinery

Look at the *wheels* row. **Vehicle** says 4 and **Bicycle** says 2. The search walks upward and stops at **Bicycle**, so it never reaches **Vehicle**’s default at all.

Overriding is not an extra feature bolted on. It falls straight out of “search upward, stop early”.

Step 4 — count the storage against a flat store

Stored here: $3 + 3 + 2 + 2 = 10$ slot entries across four frames (counting each *is-a* as one).

Flat, with every value written on **bike_9** itself: 5 entries for this one bicycle — but then every other bicycle repeats *wheels*, *engine* and *legal-on-road* too. With 20 bicycles the flat store needs $20 \times 5 = 100$ entries; the frame store needs $3 + 3 + 2 + 20 \times 2 = 48$.

Step 5 — the cost side

legal-on-road took three links to resolve. In a flat store it would take zero. That is the trade in one number: every level of hierarchy you add to save storage adds a link to every query that has to pass through it.

What it tells you

Two habits to take into an exam. First, always answer “which frame did the value come from?” as well as “what is the value?” — the marks are usually in the first half. Second, count links from the queried frame, not from the top: a slot on the object itself costs 0.

And note what has quietly happened with *engine*. From **Vehicle** alone you would conclude **bike_9** has an engine. Adding the more specific fact that it is a bicycle *removes* that conclusion. That is non-monotonic reasoning, and question D2 asks about it directly.

Questions

- D1 Counting the saving** A knowledge base has a general class **Animal** with $p = 5$ shared properties, $s = 3$ subclasses (**Bird**, **Fish**, **Mammal**), and 4 instances under each subclass.
- (a) Count the assertions needed flat and with inheritance, and give the percentage saved.
- (b) **Bird** now gains 2 further properties shared by its four instances alone. Recount both totals and the saving.
- (c) Which of the two stores is cheaper to *correct* when a shared property turns out to be wrong, and by how many edits?

Solution.

(a) *The base case*, $n = 3 \times 4 = 12$

$$\text{flat} = n \times p = 12 \times 5 = \mathbf{60}$$

$$\text{inherited} = p + s + n = 5 + 3 + 12 = \mathbf{20}$$

$$\text{saving} = \frac{60 - 20}{60} = \frac{40}{60} = \mathbf{66.67\%}$$

(b) *With Bird’s two extra properties*

Flat: the four birds each carry the five general properties *and* the two bird ones; the other eight instances carry five each.

$$\text{flat} = 12(5) + 4(2) = 60 + 8 = \mathbf{68}$$

Inherited: the two new properties are asserted once, at Bird. Nothing else changes.

$$\text{inherited} = 5 + 2 + 3 + 12 = \mathbf{22}$$

$$\text{saving} = \frac{68 - 22}{68} = \frac{46}{68} = \mathbf{67.65\%}$$

Note which way the percentage moved. Adding a property at an intermediate node made inheritance *relatively* better, not worse, because the flat store paid four times for what the inherited store paid once.

(c) *Cost of a correction*

Flat: the property is written out once per instance, so all **12** copies must be found and edited. Inherited: it is written once, so **1** edit.

The count understates the difference. Twelve edits is not twelve times the risk of one edit — it is one edit plus eleven chances to miss a copy, and a missed copy leaves the base asserting two incompatible things with no error raised. Single-point-of-truth is the real argument for inheritance; the storage saving is the one that is easy to measure.

D2 Resolving slots through a frame hierarchy

Bird	can-fly: yes (<i>default</i>), feathers: yes, legs: 2
Penguin	is-a: Bird, can-fly: no , swims: yes
tweety	is-a: Penguin, name: Tweety

(a) Give the value of *can-fly*, *legs*, *feathers* and *swims* for **tweety**, and the number of parent links traversed to find each. (b) Total the links. What would the total have been if every value were stored directly on **tweety**? (c) Learning that Tweety is a penguin *removed* a conclusion that was previously derivable. Name the property of ordinary logic this violates, and say why frame systems are willing to give it up.

Solution.

(a) *Four lookups*

slot	value	links	found at
can-fly	no	1	Penguin — — — <i>theoverride</i>
swims	yes	1	Penguin
legs	2	2	Bird
feathers	yes	2	Bird

The *can-fly* row is the one to study. The search stops at the first frame that supplies a value, so Penguin's **no** is found before Bird's default **yes** is ever consulted. Overriding is not a special mechanism — it falls out of searching upward and stopping early.

(b) *The totals*

$$1 + 1 + 2 + 2 = \mathbf{6} \text{ links traversed}$$

Stored directly on **tweety**, all four would be found at 0 links — but then every other bird would need its own copy of *legs* and *feathers*, which is precisely the flat store counted in D1.

(c) *What is given up*

Monotonicity. In ordinary logic, adding a fact can only add conclusions, never withdraw one: if $KB \models \alpha$ then $KB \cup \{\beta\} \models \alpha$ for any β .

Here, knowing only that Tweety is a bird yields $\text{can-fly} = \text{yes}$. Adding the further fact that Tweety is a penguin **retracts** it. That is non-monotonic reasoning, and plain propositional or predicate logic cannot do it.

Frame systems accept the loss because the world is like this and the logic is not. Almost every useful generalisation about the real world is a default with exceptions, and a representation that can only state exceptionless rules can state very little. The price is real — you lose the guarantee that a conclusion once drawn stays drawn — and it is why defaults sit in the representation layer rather than being smuggled into the logic.

D3 Building a network and counting what it derives Represent these six sentences as a semantic network:

- (i) Tom is a cat.
- (ii) A cat is a mammal.
- (iii) A mammal is an animal.
- (iv) Every mammal has fur.
- (v) Every animal breathes.
- (vi) Tom has a collar.

(a) How many nodes and how many edges does the network have? (b) Answer “does Tom breathe?” and “does Tom have fur?”, giving the number of edges traversed for each. (c) How many facts are *stored*? List the extra facts that are *derivable*, separating the inherited properties from the transitive IS-A facts, and give the ratio of total to stored.

Solution.

(a) *Nodes and edges*

Nodes, one per concept: Tom, Cat, Mammal, Animal, Fur, Breathing, Collar = **7** nodes.

Edges, one per sentence:

Tom $\xrightarrow{\text{is-a}}$ Cat	Mammal $\xrightarrow{\text{has}}$ Fur
Cat $\xrightarrow{\text{is-a}}$ Mammal	Animal $\xrightarrow{\text{can}}$ Breathing
Mammal $\xrightarrow{\text{is-a}}$ Animal	Tom $\xrightarrow{\text{has}}$ Collar

= **6** edges. Six sentences in, six edges out — the representation stores nothing twice.

(b) *Two queries*

Does Tom breathe? Tom $\rightarrow$ Cat $\rightarrow$ Mammal $\rightarrow$ Animal $\rightarrow$ Breathing: **4 edges**, answer **yes**.

Does Tom have fur? Tom $\rightarrow$ Cat $\rightarrow$ Mammal $\rightarrow$ Fur: **3 edges**, answer **yes**.

Neither answer is stored anywhere. Both are computed by walking, and the walk gets longer as the taxonomy gets deeper — the lookup cost from the concept section, made concrete.

(c) *Stored against derivable*

Stored: **6**.

Derivable by *inheriting a property* down the chain:

Cat has fur	Cat breathes
Tom has fur	Tom breathes
Mammal breathes	

= 5 facts.

Derivable because IS-A is *transitive*:

Tom is-a Mammal, Tom is-a Animal, Cat is-a Animal

= 3 facts.

$$\text{total available} = 6 + 5 + 3 = \mathbf{14}, \quad \frac{14}{6} = \mathbf{2.33\times} \text{ what was written down}$$

What it tells you

On a base this small the leverage is modest. It is worth seeing where the leverage actually comes from: not the six sentences, but the *depth of the chain*.

Add one more level (Tom is a Siamese, a Siamese is a Cat) and every property above it becomes derivable for the new node at the cost of a single edge. Add a hundred cats and each inherits fur and breathing for one IS-A link apiece, against two stored assertions each in a flat base.

More practice Answers are given in brackets.

- i. $p = 4$ shared properties, $s = 5$ subclasses, $n = 40$ instances. Give the flat and inherited counts, and the percentage saved. [**flat** 160; **inherited** $4 + 5 + 40 = 49$; **saved** 69.38%]
- ii. In the same base, one subclass is an exception to one property. How many extra assertions? [1]
- iii. a is-a B, B is-a C, C is-a D. A slot is defined only on D. How many links does a query on a walk? [3]
- iv. Same chain, but B also defines that slot. How many links now, and which value wins? [1 **link; B's value — the search stops early**]
- v. Which representation handles “birds fly, except penguins” more naturally, and why? [**frames — a child slot simply overrides the parent's default**]

6 Part E — Production systems and logic

1. The concept

A production rule IF $\langle \text{conditions} \rangle$ THEN $\langle \text{conclusion or action} \rangle$. The conditions are matched against the facts currently held. Nothing more. The whole expressive power of an expert system is a list of these.

The three components a **knowledge base** of rules, a **working memory** of facts believed right now, and an **inference engine**. The engine repeats a *recognise-act cycle*:

- (1) find every rule whose conditions are satisfied — the **conflict set**;
- (2) choose one — this is **conflict resolution**;
- (3) fire it, and add its conclusion to working memory.

Conflict resolution is done by rule order, by recency of the matched facts, or by specificity — most conditions wins.

Forward chaining start from the facts and fire whatever applies, adding conclusions until nothing new appears. *Data-driven*: it derives everything derivable, whether you wanted it or not.

Backward chaining start from a hypothesis and work backwards. Ask what would have to be true for it to hold, recurse on each condition, and ask the user only when no rule can supply an answer. *Goal-driven*, and the natural choice for diagnosis — where the point is to reach one conclusion while asking as few questions as possible.

Why logic, then because rules alone give no account of what follows from what. Propositional logic makes that precise. A sentence is **valid** if true in every row of its truth table, **satisfiable** if true in at least one row, and $KB \models \alpha$ when every row satisfying KB also satisfies α . First-order predicate logic adds objects, predicates and quantifiers, so a rule can be stated once for all objects instead of once per object.

2. The picture

Figure 6 shows the same rule base driven from both ends. The two arrows run in opposite directions, and so does the work.

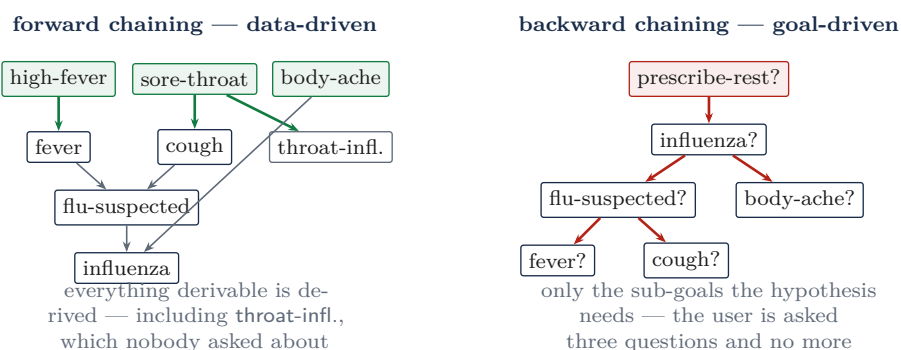


Figure 6: One rule base, two directions. Forward chaining fills working memory; backward chaining builds a proof tree for a single hypothesis. Which is cheaper depends entirely on whether you have one question or many.

3. How it differs — forward against backward chaining

	Forward chaining	Backward chaining
Starts from	the known facts	the hypothesis to be proved
Driven by	data	goals
Stops when	no rule adds anything new	the goal is proved or every route fails
Derives	everything derivable, relevant or not	only what the goal depends on
Asks the user	nothing — it works from what it has	only for facts no rule can supply
Natural fit	monitoring, alarms, “what follows from this?”	diagnosis, “is it X ?”

The row on what gets derived is the one with the cost attached. A monitoring system *wants* every consequence of a new sensor reading. A diagnostic system does not. Asking a patient forty questions to establish forty facts, thirty-seven of which have no bearing on the hypothesis, is not a performance problem — it is a usability failure.

4. Key points

Key points — remember these

- A production system = rules + working memory + inference engine.
 - The cycle is: match (build the conflict set) $\rightarrow$ resolve (choose one rule) $\rightarrow$ fire (add the conclusion).
 - **Forward** chaining is data-driven; **backward** chaining is goal-driven. Neither is “better” — it depends on whether you arrived with facts or with a question.
 - A rule enters the conflict set only when **all** its conditions are present. Half a match is no match.
 - Conflict resolution strategies: rule order, recency, specificity. For safety systems, prefer **specificity** — the narrower rule encodes the exception.
 - Truth table rows = 2^n . Valid = true in every row; satisfiable = true in at least one; $KB \models \alpha$ = every row that makes KB true makes α true.
 - To disprove entailment you need **one** counterexample row. To prove it you need all of them. That asymmetry is why the method is exhaustive.
 - An unsatisfiable KB entails everything. A contradiction is not a small bug.
 - $\forall$ pairs with $\rightarrow$; $\exists$ pairs with $\wedge$. Mixing them up is the most common translation error.
 - Propositional encoding of a domain needs $pn + qn^2$ symbols and 2^{pn+qn^2} rows. That is the argument for quantifiers.
-

5. The numericals

Worked example 6.1 — *running the same rule base both ways*

R_1 : IF fever $\wedge$ cough THEN flu-suspected
 R_2 : IF flu-suspected $\wedge$ body-ache THEN influenza
 R_3 : IF sore-throat THEN cough
 R_4 : IF influenza THEN prescribe-rest
 R_5 : IF high-fever THEN fever
 R_6 : IF sore-throat THEN throat-inflammation
 R_7 : IF throat-inflammation THEN gargle-advice

Working memory starts as {high-fever, sore-throat, body-ache}. Conflict resolution: fire the lowest-numbered rule that adds a fact not already present.

Step 1 — forward chaining, cycle by cycle

cycle	conflict set	fires, adding
1	R_3, R_5, R_6	$R_3 \Rightarrow$ cough
2	R_5, R_6	$R_5 \Rightarrow$ fever
3	R_1, R_6	$R_1 \Rightarrow$ flu-suspected
4	R_2, R_6	$R_2 \Rightarrow$ influenza
5	R_4, R_6	$R_4 \Rightarrow$ prescribe-rest
6	R_6	$R_6 \Rightarrow$ throat-inflammation
7	R_7	$R_7 \Rightarrow$ gargle-advice
8	empty	— halt

Cycle 1 is worth checking. R_1 needs fever *and* cough, and neither is present yet, so R_1 is not in the conflict set. A rule joins the conflict set only when *all* of its conditions are satisfied.

Step 2 — total the forward-chaining work

rules fired = **7**, cycles run = **8** (the last one finds nothing)

final working memory = 3 given + 7 derived = **10** facts

Step 3 — backward chaining for the goal prescribe-rest

sub-goal	resolved by
prescribe-rest	$R_4 \Rightarrow$ need influenza
influenza	$R_2 \Rightarrow$ need flu-suspected $\wedge$ body-ache
flu-suspected	$R_1 \Rightarrow$ need fever $\wedge$ cough
fever	$R_5 \Rightarrow$ need high-fever; ask the user $\Rightarrow$ yes
cough	$R_3 \Rightarrow$ need sore-throat; ask the user $\Rightarrow$ yes
body-ache	no rule concludes it; ask the user $\Rightarrow$ yes

rules used = **5** (R_4, R_2, R_1, R_5, R_3), questions asked = **3**

Step 4 — compare

$$\frac{\text{forward firings}}{\text{backward rule uses}} = \frac{7}{5} = 1.4 \quad \text{and } R_6, R_7 \text{ were never touched backwards.}$$

What it tells you

Two of forward chaining's seven firings — *throat-inflammation* and *gargle-advice* — contributed nothing to the goal. That is not a bug. It is the definition of data-driven. Forward chaining does not know what you want, so it derives everything, and on a base of five hundred rules the two wasted firings become four hundred.

Backward chaining touched exactly the five rules the hypothesis depends on. The number that matters most in a real deployment is the last one: **three questions**. Forward chaining

would have needed every relevant fact supplied up front, before it knew which were relevant — which is why diagnosis is backward-chained and monitoring is forward-chained, and why MYCIN was built the way it was.

Worked example 6.2 — checking an argument with a truth table

Is *modus ponens*, $((P \rightarrow Q) \wedge P) \rightarrow Q$, valid? And is *affirming the consequent*, $((P \rightarrow Q) \wedge Q) \rightarrow P$?

Step 1 — count the rows

Two symbols, so $2^2 = 4$ rows. Enumerate all of them. A truth table is only convincing if it is exhaustive.

Step 2 — build the table

P	Q	$P \rightarrow Q$	$(P \rightarrow Q) \wedge P$	$((P \rightarrow Q) \wedge P) \rightarrow Q$	$(P \rightarrow Q) \wedge Q$	$((P \rightarrow Q) \wedge Q) \rightarrow P$
T	T	T	T	T	T	T
T	F	F	F	T	F	T
F	T	T	F	T	T	F
F	F	T	F	T	F	T

Step 3 — read the two conclusion columns

Column five is T in all four rows, so *modus ponens* is **valid**.

Column seven is F in row three, so affirming the consequent is **invalid**. Row three is the counterexample: P false, Q true. Both premises hold — $P \rightarrow Q$ is true because P is false, and Q is true — while the conclusion P is false.

What it tells you

One row is enough to destroy a claim of validity, and no number of rows short of all of them is enough to establish it. That asymmetry is why the method is exhaustive enumeration rather than sampling.

It is also worth naming the fallacy in words, because it is the single most common error in rule-based reasoning: “*if flu then fever*; the patient has fever; therefore flu”. The rule licenses no such thing. Something else causes fever, and row three of the table is exactly that patient.

Worked example 6.3 — expanding quantifiers, and counting what they save

A domain has three objects, $\{a, b, c\}$, one unary predicate $\text{Cat}(x)$, and one binary predicate $\text{Chases}(x, y)$.

Step 1 — expand a universal into a conjunction

$\forall$ over a finite domain is a long AND, one conjunct per object:

$$\forall x \text{Cat}(x) \equiv \text{Cat}(a) \wedge \text{Cat}(b) \wedge \text{Cat}(c)$$

Step 2 — expand an existential into a disjunction

$\exists$ is a long OR:

$$\exists x \text{Cat}(x) \equiv \text{Cat}(a) \vee \text{Cat}(b) \vee \text{Cat}(c)$$

Step 3 — expand a binary predicate

Every *ordered pair* gives one conjunct, so there are $n^2 = 9$:

$$\begin{aligned} \forall x \forall y \text{Chases}(x, y) \equiv & \text{Ch}(a, a) \wedge \text{Ch}(a, b) \wedge \text{Ch}(a, c) \wedge \\ & \text{Ch}(b, a) \wedge \text{Ch}(b, b) \wedge \text{Ch}(b, c) \wedge \\ & \text{Ch}(c, a) \wedge \text{Ch}(c, b) \wedge \text{Ch}(c, c) \end{aligned}$$

Step 4 — count the propositional symbols

With p unary and q binary predicates over n objects:

$$\text{symbols} = pn + qn^2 = 1(3) + 1(9) = \mathbf{12}$$

$$\text{truth-table rows} = 2^{12} = \mathbf{4096}$$

Step 5 — now grow the domain to $n = 50$

$$\text{symbols} = 1(50) + 1(2500) = \mathbf{2550}, \quad \text{rows} = 2^{2550}$$

The two quantified sentences did not change at all. They are the same dozen characters they were for three objects.

Step 6 — the translation habits worth fixing

$$\text{“every cat chases something”} \equiv \forall x (\text{Cat}(x) \rightarrow \exists y \text{Chases}(x, y))$$

$$\text{“some cat chases nothing”} \equiv \exists x (\text{Cat}(x) \wedge \neg \exists y \text{Chases}(x, y))$$

$\forall$ pairs with $\rightarrow$. $\exists$ pairs with $\wedge$. Writing $\forall x(\text{Cat}(x) \wedge \dots)$ claims that *everything in the universe* is a cat, and $\exists x(\text{Cat}(x) \rightarrow \dots)$ is satisfied by any non-cat at all.

What it tells you

Step 5 is the whole argument for predicate logic in one line. The propositional encoding grows as n for properties and as n^2 for relations, and the truth table grows as 2 to that power. The quantified sentence does not grow.

Quantifiers are not notational convenience. They are what makes a statement independent of the size of the world.

Questions

E1 Forward and backward on one rule base

$$\begin{array}{ll} R_1 : \text{IF } A \wedge B \text{ THEN } C & R_4 : \text{IF } B \text{ THEN } E \\ R_2 : \text{IF } C \text{ THEN } D & R_5 : \text{IF } G \text{ THEN } H \\ R_3 : \text{IF } D \wedge E \text{ THEN } F \end{array}$$

Working memory starts as $\{A, B\}$; fire the lowest-numbered applicable rule that adds something new. **(a)** Trace forward chaining. Give the conflict set and the rule fired at each cycle, the order in which facts are derived, and the final working memory. **(b)** Backward-chain the goal F . Which rules are used, and how many questions must be put to the user? **(c)** Now the goal is H . Compare the two strategies on this goal.

Solution.

(a) Forward chaining

cycle	working memory	conflict set	fires
1	A, B	R_1, R_4	$R_1 \Rightarrow C$
2	A, B, C	R_2, R_4	$R_2 \Rightarrow D$
3	A, B, C, D	R_4	$R_4 \Rightarrow E$
4	A, B, C, D, E	R_3	$R_3 \Rightarrow F$
5	A, B, C, D, E, F	empty	halt

Cycle 3 is the one worth checking: R_3 needs D and E , and E is not yet present, so R_3 is not in the conflict set even though half of it matches.

$$\text{derivation order} = C, D, E, F \quad \text{rules fired} = \mathbf{4} (R_1, R_2, R_4, R_3)$$

final working memory = $\{A, B, C, D, E, F\}$ = **6** facts

R_5 never fires: G is not present and nothing derives it.

(b) *Backward chaining for F*

sub-goal	resolved by
F	$R_3 \Rightarrow$ need $D \wedge E$
D	$R_2 \Rightarrow$ need C
C	$R_1 \Rightarrow$ need $A \wedge B$; ask A, ask B
E	$R_4 \Rightarrow$ need B (already established)

rules used = **4** (R_3, R_2, R_1, R_4), questions = **2** (A and B)

B is asked once and reused for both R_1 and R_4 — a competent engine caches established sub-goals rather than re-asking.

(c) *The goal H*

Backward chaining: $H \leftarrow R_5$ needs G ; no rule concludes G , so it asks the user one question. If the answer is no, the goal fails.

rules examined = **1**, questions = **1**

Forward chaining: it cannot aim at H . It fires all four applicable rules, derives C, D, E, F — none of which bear on H — halts, and H is simply absent from working memory.

rules fired = **4**, all of them irrelevant

What it tells you

Part (c) is the clean case. For a single specific hypothesis, backward chaining does one rule's worth of work and asks one question. Forward chaining does four rules' worth and answers a different question entirely.

Reverse the situation — you have the facts and want every consequence, as a monitoring system does — and forward chaining wins just as decisively. The choice is not about which algorithm is better. It is about whether you arrived with facts or with a question.

E2 Truth tables and entailment (a) How many rows does a truth table have for 3 symbols? For 6? For 20? (b) Let $KB = \{P \vee Q, \neg Q\}$. Build the table and determine whether $KB \models P$. (c) Does $KB \models Q$? Justify from the same table. (d) Is KB satisfiable? Is $P \wedge \neg P$ satisfiable?

Solution.

(a) *Row counts, 2^n*

$$2^3 = 8, \quad 2^6 = 64, \quad 2^{20} = 1\,048\,576$$

(b) *The table for KB*

P	Q	$P \vee Q$	$\neg Q$	KB true?	P
T	T	T	F	no	T
T	F	T	T	YES	T
F	T	T	F	no	F
F	F	F	T	no	F

KB has exactly **one** model, row two, and P is true in it. Every model of KB satisfies P , so

$$KB \models P \quad \text{yes}$$

This is disjunctive syllogism, established by enumeration rather than by a named rule.

(c) Does $KB \models Q$?

No. In the single model of KB , Q is *false*. One model that satisfies KB and falsifies Q is all it takes, and here that model is the only one there is.

(d) *Satisfiability*

KB is **satisfiable** — row two makes it true, and one row is the whole requirement.

$P \wedge \neg P$ is **unsatisfiable**: it is false when P is true and false when P is false, exhausting the possibilities. An unsatisfiable KB entails *everything*, which is why a contradiction in a knowledge base is not a small problem — the system will cheerfully prove any conclusion at all.

What it tells you

Look back at $2^{20} = 1\,048\,576$ from part (a). Twenty facts about the world is nothing — a single patient record has more — and the table is already past a million rows. Truth tables are how entailment is *defined*. They are not how any real system computes it, and that gap is what the rest of automated reasoning exists to close.

E3 Finding a counterexample Let $KB = \{P \vee Q, \neg P \rightarrow R\}$ and $\alpha = Q \vee R$. **(a)** How many rows must be checked? **(b)** Does $KB \models \alpha$? If not, give the counterexample row explicitly. **(c)** A colleague argues: “either P or Q ; and if not P then R ; so surely Q or R .” Explain in words where the argument fails.

Solution.

(a) *Rows*

Three symbols P, Q, R , so $2^3 = 8$ rows.

(b) *The table*

P	Q	R	$P \vee Q$	$\neg P \rightarrow R$	KB	$Q \vee R$
T	T	T	T	T	yes	T
T	T	F	T	T	yes	T
T	F	T	T	T	yes	T
T	F	F	T	T	yes	F
F	T	T	T	T	yes	T
F	T	F	T	F	no	—
F	F	T	F	T	no	—
F	F	F	F	F	no	—

$$KB \not\models \alpha$$

The counterexample is row four: **P = T, Q = F, R = F**. Check it directly — $P \vee Q$ is true because P is true; $\neg P$ is false, so $\neg P \rightarrow R$ is true *vacuously*; so KB holds. And $Q \vee R$ is $F \vee F = F$. One row is sufficient.

(c) *Where the argument fails*

On the word “*surely*”. The second premise is a *conditional*, and a conditional whose antecedent is false tells you nothing at all — it is automatically true and makes no claim about R .

When P holds, the first premise is already satisfied by P alone, so Q need not hold; and $\neg P$ fails, so the second premise never fires and R need not hold either. Both premises are true and both disjuncts of the conclusion are free to be false.

The colleague has quietly read the second premise as “ R is true” rather than “ R is true *if* P is not”. That is the same misuse of an implication that the worked example above diagnosed as affirming the consequent, and it is why the table is worth building even when the argument sounds obviously right.

E4 What quantifiers are worth A domain contains four objects, $\{a, b, c, d\}$. **(a)** Write out $\forall x (\text{Student}(x) \rightarrow \text{Studies}(x))$ and $\exists x (\text{Student}(x) \wedge \neg \text{Studies}(x))$ as fully propositional sentences over this domain. **(b)** How many conjuncts would $\forall x \forall y \text{Knows}(x, y)$ expand to? **(c)** With 3 unary and 2 binary predicates, how many propositional symbols encode a domain of 4 objects? Of 100 objects? Give the truth-table size in the second case. **(d)** Translate: “every student who studies passes” and “some student does not study”.

Solution.

(a) The expansions

$\forall$ becomes a conjunction over the domain:

$$(\text{St}(a) \rightarrow \text{Sd}(a)) \wedge (\text{St}(b) \rightarrow \text{Sd}(b)) \wedge (\text{St}(c) \rightarrow \text{Sd}(c)) \wedge (\text{St}(d) \rightarrow \text{Sd}(d))$$

$\exists$ becomes a disjunction:

$$(\text{St}(a) \wedge \neg \text{Sd}(a)) \vee (\text{St}(b) \wedge \neg \text{Sd}(b)) \vee (\text{St}(c) \wedge \neg \text{Sd}(c)) \vee (\text{St}(d) \wedge \neg \text{Sd}(d))$$

Four objects, four conjuncts or disjuncts.

(b) A binary predicate

Every ordered pair (x, y) gives one conjunct:

$$n^2 = 4^2 = \mathbf{16} \text{ conjuncts}$$

(c) Encoding cost

$$\text{symbols} = p n + q n^2$$

For $n = 4$: $3(4) + 2(16) = 12 + 32 = \mathbf{44}$ symbols.

For $n = 100$: $3(100) + 2(10\,000) = 300 + 20\,000 = \mathbf{20\,300}$ symbols, and a truth table of

$$2^{20\,300} \text{ rows}$$

— a number with more than six thousand digits, for a domain of a hundred things and five predicates.

(d) Translations

$$\forall x ((\text{Student}(x) \wedge \text{Studies}(x)) \rightarrow \text{Passes}(x))$$

$$\exists x (\text{Student}(x) \wedge \neg \text{Studies}(x))$$

Two habits are worth fixing here. $\forall$ pairs with $\rightarrow$ and $\exists$ pairs with $\wedge$. Writing $\forall x (\text{St}(x) \wedge \text{Sd}(x))$ claims that *everything in the universe* is a studying student, and $\exists x (\text{St}(x) \rightarrow \neg \text{Sd}(x))$ is satisfied by any non-student at all.

What it tells you

Part (c) is the argument for predicate logic in one line. The quantified sentence in (d) is a dozen symbols long, and its meaning does not change when the university admits another thousand students.

The propositional encoding of the same fact grows as n , or as n^2 once relations are involved, and its truth table grows as 2 to that power. Quantifiers are not notational convenience. They are what makes the statement independent of the size of the world.

E5 Conflict resolution Working memory holds three facts, with the cycle each was added on:

pressure-high (added cycle 1), valve-open (cycle 2), temp-high (cycle 3)

and three rules all match:

R_1 : IF temp-high THEN sound-alarm
 R_2 : IF temp-high $\wedge$ pressure-high THEN shut-down
 R_3 : IF valve-open THEN log-event

(a) Which rule fires under rule-order, under recency, and under specificity? (b) Which strategy would you choose for a plant safety controller, and why? (c) What is the size of the conflict set, and what happens to it after the chosen rule fires?

Solution.

(a) *The three strategies*

Rule order — lowest number first: **R_1** , sound-alarm.

Recency — prefer the rule matching the most recently added fact. The newest fact is *temp-high* (cycle 3), matched by both R_1 and R_2 , so recency alone does not decide. The usual tie-break is specificity, giving **R_2** .

Specificity — most conditions wins. R_1 has 1 condition, R_2 has **2**, R_3 has 1. So **R_2** , shut-down.

(b) *For a safety controller*

Specificity. The reasoning is not about efficiency at all. R_2 describes a strictly narrower and more dangerous situation than R_1 : high temperature *together with* high pressure. R_1 matches whenever R_2 does, so if the more general rule is allowed to fire first, the system sounds an alarm on a plant that needed shutting down — and every extra condition an engineer wrote into R_2 was wasted.

The general principle: when one rule's conditions are a superset of another's, the specific rule encodes the exception, and firing the general one discards the exception.

(c) *The conflict set*

All three rules match, so the conflict set has **3** members.

After R_2 fires, *shut-down* enters working memory. R_2 leaves the conflict set — refiring it would add nothing new — and R_1 and R_3 remain, so the set is now **2**.

It shrinks by one per cycle here only because no rule's conclusion enables another. In the worked example above, firings created new matches and the set could grow.

What it tells you

The rule base is identical in all three cases, and the outcome is alarm, shutdown or a log entry depending purely on the conflict-resolution strategy.

The inference engine's policy is part of the system's behaviour, not an implementation detail — and it is invisible in the rules themselves, which is exactly what makes it easy to get wrong.

More practice Answers are given in brackets.

- i. Working memory is $\{X\}$, and the rules are R_1 : IF X THEN Y ; R_2 : IF $Y \wedge Z$ THEN W ; R_3 : IF Y THEN Z . Trace forward chaining. [$R_1 \Rightarrow Y$, $R_3 \Rightarrow Z$, $R_2 \Rightarrow W$; **final** $\{X, Y, Z, W\}$]
- ii. How many rows does a truth table have for 5 symbols? For 10? [**32 and 1024**]
- iii. Is $P \rightarrow (Q \rightarrow P)$ valid? [**yes — it is true in all four rows**]
- iv. $KB = \{P \rightarrow Q, Q\}$. Does $KB \models P$? [**no — P false, Q true satisfies KB and falsifies P ; this is affirming the consequent**]
- v. A domain of 6 objects with 2 unary and 1 binary predicate. How many propositional symbols? [$2(6) + 1(36) = 48$]
- vi. Translate “no student fails”. [$\forall x (\text{Student}(x) \rightarrow \neg \text{Fails}(x))$]

7 Part F — Reasoning under uncertainty with Bayes' theorem

1. The concept

Why the rules of Part E are not enough a production rule is certain: IF fever THEN flu asserts the conclusion outright. Real diagnosis is not like that. Fever raises the probability of flu; it does not establish it. Bayes' theorem is the arithmetic of that revision.

The four quantities the **prior** $P(H)$ is what you believed before seeing anything — typically the base rate of the condition in the population. The **likelihood** $P(E | H)$ says how well the hypothesis explains the evidence; for a medical test, this is its sensitivity. The **evidence** $P(E)$ is the total probability of seeing E at all, from any cause. The **posterior** $P(H | E)$ is the revised belief.

Base-rate neglect the single most common error, and the reason this topic is examined. A test with 90% sensitivity feels like 90% certainty after a positive result. It is not, and it is not close, whenever the condition is rare. The enormous healthy majority contributes a flood of false positives that can swamp the true ones. The prior is not a detail to be mentioned in passing. It frequently dominates the answer.

The natural-frequency trick the whole calculation becomes obvious if you stop using percentages and imagine a concrete population. Take 10 000 people, count how many are ill, count how many of each group test positive, and read the answer straight off. It is the same arithmetic, and it is much harder to get wrong.

2. The picture

Figure 7 is the first worked example drawn as natural frequencies. The answer is visible before any formula is applied: compare the size of the two boxes at the bottom.

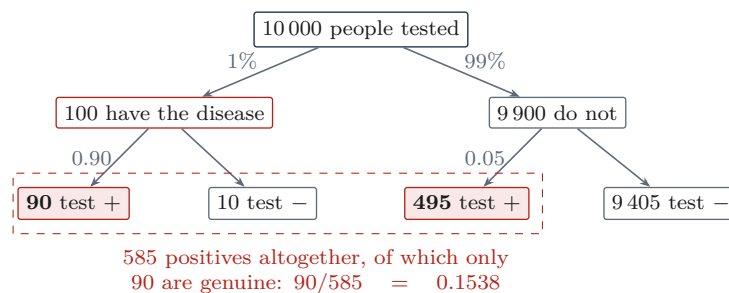


Figure 7: The same numbers as the formula, laid out as people. The 495 false positives come from a group 99 times larger than the sick one — which is why a low base rate defeats a good test, and why the answer is 15% rather than the 90% the sensitivity suggests.

3. How it differs — sensitivity against the posterior

	Sensitivity $P(+ D)$	Posterior $P(D +)$
Question it answers	of the people who are ill, how many test positive?	of the people who test positive, how many are ill?
Depends on the base rate?	no — it is a property of the test	yes , heavily
Who needs it	the person validating the test	the patient holding the result
In the worked example	0.90	0.1538

The two are constantly confused, and the confusion has a name: they are *conditional probabilities with the condition the wrong way round*. $P(+ | D)$ and $P(D | +)$ are different numbers about different populations. Bayes' theorem is precisely the rule for converting one into the other, and the converting factor is the prior.

4. Key points

Key points — remember these

- $P(H | E) = \frac{P(E | H)P(H)}{P(E)}$, with $P(E) = P(E | H)P(H) + P(E | \neg H)P(\neg H)$.
 - The denominator is **every** way the evidence could arise — true positives *plus* false positives. Forgetting the second term is the commonest arithmetic slip.
 - $P(+ | D)$ and $P(D | +)$ are different numbers. Sensitivity is a property of the test; the posterior is a property of the test *and* the population.
 - **Base-rate neglect**: when the condition is rare, even an excellent test gives a low posterior. Always check the prior first.
 - Natural frequencies are the safest method. Take 10 000 people and count. Use it to check every formula answer.
 - Odds form: posterior odds = prior odds $\times$ likelihood ratio, then $P = \text{odds} / (1 + \text{odds})$. Several pieces of evidence just multiply.
 - For rare conditions the **false-positive rate** governs the answer, not the sensitivity. Improving specificity is what helps.
 - Naive Bayes: score each hypothesis as $P(H) \prod_i P(E_i | H)$, then normalise so the scores sum to 1. This assumes conditional independence, which is usually false, and makes posteriors too extreme.
-

5. The numericals

Worked example 7.1 — reading a positive screening test

A disease affects 1% of a population. The test has sensitivity $P(+ | D) = 0.90$ and a false-positive rate $P(+ | \neg D) = 0.05$. A patient tests positive. What is the probability they have the disease?

Step 1 — write down what is given, and the complement

$$P(D) = 0.01, \quad P(\neg D) = 1 - 0.01 = 0.99$$

$$P(+ | D) = 0.90, \quad P(+ | \neg D) = 0.05$$

Step 2 — the numerator: sick and positive

$$P(+ | D) P(D) = 0.90 \times 0.01 = 0.009$$

Step 3 — the denominator: positive from any cause

$$P(+) = \underbrace{0.90 \times 0.01}_{\text{true positives}} + \underbrace{0.05 \times 0.99}_{\text{false positives}} = 0.009 + 0.0495 = 0.0585$$

Step 4 — divide

$$P(D | +) = \frac{0.009}{0.0585} = \mathbf{0.1538} \quad (15.38\%)$$

Step 5 — check it against Figure 7

Out of 10 000: 100 are ill and 90 of them test positive; 9 900 are well and 495 of them test positive. Of 585 positives, 90 are genuine:

$$\frac{90}{585} = 0.1538 \quad \checkmark$$

Step 6 — the same answer in odds form

$$\text{prior odds} = \frac{0.01}{0.99} = \frac{1}{99}, \quad \text{LR} = \frac{0.90}{0.05} = 18$$

$$\text{posterior odds} = \frac{1}{99} \times 18 = \frac{18}{99} = \frac{2}{11}$$

$$P = \frac{2/11}{1 + 2/11} = \frac{2}{13} = \mathbf{0.1538} \quad \checkmark$$

What it tells you

The positive result multiplied the odds by 18. The test is genuinely informative, and belief rose from 1% to 15.38% — a fifteenfold increase. And the patient still probably does not have the disease.

Both halves of that sentence are true at once, and holding them together is the skill. The test is good. The prior was tiny. $18 \times$ a tiny number is still a small number.

The practical consequence is the one screening programmes actually implement: a positive result is not a diagnosis. It is a reason to run a second, independent test on a population whose base rate is now 15% instead of 1%.

Worked example 7.2 — two hypotheses, two symptoms

A diagnostic system considers exactly two possibilities, Bacterial and Viral, with priors

$$P(\text{Bact}) = 0.20, \quad P(\text{Viral}) = 0.80$$

and these likelihoods:

	Bact	Viral
$P(\text{high-fever} \cdot)$	0.70	0.30
$P(\text{raised-WBC} \cdot)$	0.80	0.20

A patient has *both* symptoms. The symptoms are assumed conditionally independent given the disease.

Step 1 — score each hypothesis: prior $\times$ all its likelihoods

Do *not* normalise yet. These scores are $P(H, E_1, E_2)$, not probabilities of H .

$$\text{score}(\text{Bact}) = 0.20 \times 0.70 \times 0.80 = \mathbf{0.112}$$

$$\text{score}(\text{Viral}) = 0.80 \times 0.30 \times 0.20 = \mathbf{0.048}$$

Step 2 — normalise, so the two add to 1

$$\text{total} = 0.112 + 0.048 = 0.160$$

$$P(\text{Bact} \mid E) = \frac{0.112}{0.160} = \mathbf{0.7000}, \quad P(\text{Viral} \mid E) = \frac{0.048}{0.160} = \mathbf{0.3000}$$

The total 0.160 is $P(E)$, the probability of seeing both symptoms at all. Dividing by it is exactly the denominator of Bayes' theorem — computed as a by-product rather than as a separate step.

Step 3 — the same result in odds form, as a check

$$\text{prior odds (Bact : Viral)} = \frac{0.20}{0.80} = \frac{1}{4}$$

$$\text{combined LR} = \frac{0.70}{0.30} \times \frac{0.80}{0.20} = 2.3333 \times 4 = 9.3333$$

$$\text{posterior odds} = \frac{1}{4} \times 9.3333 = 2.3333 \implies P = \frac{2.3333}{1 + 2.3333} = \mathbf{0.7000} \quad \checkmark$$

Step 4 — what one symptom alone would have given

With high fever only:

$$\text{odds} = \frac{1}{4} \times \frac{0.70}{0.30} = 0.5833 \implies P(\text{Bact}) = \frac{0.5833}{1.5833} = \mathbf{0.3684}$$

So the first symptom moved belief from 0.20 to 0.3684, and the second from 0.3684 to 0.7000. Evidence accumulates by multiplying likelihood ratios, one at a time or all together — the answer is the same.

Step 5 — name the assumption

Conditional independence means: *once you know the disease*, learning the patient has a high fever tells you nothing further about the white-cell count.

$$P(\text{WBC} \mid \text{Bact, fever}) = P(\text{WBC} \mid \text{Bact})$$

That is what licenses multiplying 0.70 by 0.80. It is very likely false here, since both symptoms follow from the same inflammatory response.

What it tells you

The method is short enough to memorise: *score, then normalise*. It works for any number of hypotheses, as long as they are exhaustive and mutually exclusive — which is what lets the scores be turned into probabilities by dividing by their total.

The characteristic weakness is worth recognising in an exam answer. Because the independence assumption double-counts overlapping evidence, naive Bayes produces posteriors that are **too extreme** — 0.70 where the truth might be 0.60. Its *ranking* of hypotheses is usually still right, which is why the method survives an assumption that is almost never true.

Questions

F1 A fault-detection alarm A fault occurs in 2% of devices. The alarm detects a genuine fault with probability 0.95, and false-alarms on a healthy device with probability 0.04. The alarm has just sounded. **(a)** Compute $P(\text{fault} \mid +)$. **(b)** Verify it with natural frequencies out of 10 000 devices. **(c)** What fraction of all alarms are false? What does that imply for the technician who has to respond to each one?

Solution.

(a) *Bayes' theorem*

Let F be “faulty”. $P(F) = 0.02$, so $P(\neg F) = 0.98$.

$$P(+) = \underbrace{0.95 \times 0.02}_{0.019} + \underbrace{0.04 \times 0.98}_{0.0392} = 0.0582$$

$$P(F \mid +) = \frac{0.019}{0.0582} = \mathbf{0.3265} \quad (32.65\%)$$

(b) *Natural frequencies, out of 10 000*

	devices	alarm +	alarm −
faulty	200	190	10
healthy	9 800	392	9 408
total	10 000	582	9 418

$$\frac{190}{582} = \mathbf{0.3265} \quad \checkmark$$

(c) *The false-alarm burden*

$$\frac{392}{582} = \mathbf{0.6735} \quad (67.35\% \text{ of alarms are false})$$

Roughly two in every three call-outs find nothing wrong. That is not a hypothetical inconvenience — it is how alarm systems get switched off. An operator who learns that a sounding alarm usually means nothing begins to treat it as noise, and the system's genuine detections go unanswered along with the rest.

The fix is not a better sensitivity; 0.95 is already high. It is a lower false-positive rate, or a confirmatory second check before a human is summoned.

F2 A second, independent test The same device from F1, having alarmed once, is tested again with an independent alarm of identical characteristics (0.95 and 0.04), and it alarms again. **(a)** Compute $P(\text{fault} \mid ++)$ using the F1 posterior as the new prior. **(b)** Repeat the calculation in odds form, starting from the original 2%. **(c)** Summarise the three belief states in one line, and say what the word “independent” is doing in this question.

Solution.

(a) *Chaining the update*

The prior is now $P(F) = 0.3265$, so $P(\neg F) = 0.6735$.

$$P(+_2) = 0.95(0.3265) + 0.04(0.6735) = 0.310175 + 0.026940 = 0.337115$$

$$P(F \mid ++) = \frac{0.310175}{0.337115} = \mathbf{0.9201} \quad (92.01\%)$$

(b) *The odds route, in one line*

$$\begin{aligned}\text{LR} &= \frac{0.95}{0.04} = 23.75, & \text{prior odds} &= \frac{0.02}{0.98} = \frac{1}{49} \\ \text{posterior odds} &= \frac{1}{49} \times 23.75 \times 23.75 = \frac{564.0625}{49} = 11.5115 \\ P &= \frac{11.5115}{1 + 11.5115} = \mathbf{0.9201} \quad \checkmark\end{aligned}$$

Two updates are two multiplications. That is the practical argument for the odds form: evidence accumulates by multiplying likelihood ratios, and the normalisation is done once, at the end.

(c) *The three states, and the assumption*

$$2\% \longrightarrow 32.65\% \longrightarrow 92.01\%$$

Independent is carrying the entire argument. The calculation assumes the second alarm's error is unrelated to the first's — that $P(+_2 \mid F, +_1) = P(+_2 \mid F)$.

If instead both alarms share a sensor, a power supply, or a calibration fault, then whatever made the first one false-alarm makes the second one false-alarm too. The second reading then carries almost no new information, and 92% is badly overstated.

This is exactly why confirmatory tests are chosen to work on a *different principle* from the first, rather than simply being the same test run twice.

F3 The same test, a different population The F1 alarm (0.95, 0.04) is now deployed on ageing equipment where 30% of devices are faulty. **(a)** Compute $P(F \mid +)$. **(b)** Compare with the 0.3265 from F1 and state precisely what changed. **(c)** At what base rate does a single alarm make the fault exactly as likely as not?

Solution.

(a) *The high-prevalence case*

$$\begin{aligned}P(+) &= 0.95(0.30) + 0.04(0.70) = 0.285 + 0.028 = 0.313 \\ P(F \mid +) &= \frac{0.285}{0.313} = \mathbf{0.9105} \quad (91.05\%)\end{aligned}$$

(b) *What changed*

	base rate 2%	base rate 30%
sensitivity	0.95	0.95
false-positive rate	0.04	0.04
likelihood ratio	23.75	23.75
$P(F \mid +)$	0.3265	0.9105

The test did not change. Not one of its properties changed. The same device, giving the same reading, from the same instrument, supports a conclusion that is 32.65% likely in one setting and 91.05% likely in the other — purely because of the population it was drawn from.

(c) *The break-even base rate*

Fifty-fifty means posterior odds of exactly 1:

$$\text{prior odds} \times 23.75 = 1 \implies \text{prior odds} = \frac{1}{23.75}$$

$$P = \frac{1/23.75}{1 + 1/23.75} = \frac{1}{24.75} = \mathbf{0.0404} \quad (4.04\%)$$

Below a 4.04% base rate, a single alarm still leaves the fault *less* likely than not.

What it tells you

This is the whole of base-rate neglect in three numbers. The instinct that a test's accuracy determines what its result means is simply wrong. Accuracy fixes the likelihood ratio, and the likelihood ratio is only *half* the calculation.

Ask what population the case came from before interpreting any result — and notice that F1's device and F3's device may be physically identical.

F4 Two symptoms, two hypotheses A diagnostic system considers exactly two possibilities, with priors $P(\text{Flu}) = 0.10$ and $P(\text{Cold}) = 0.90$. The likelihoods are

	Flu	Cold
$P(\text{fever} \mid \cdot)$	0.90	0.20
$P(\text{ache} \mid \cdot)$	0.80	0.10

A patient presents with both fever and body-ache. Assume the two symptoms are conditionally independent given the disease. **(a)** Compute the unnormalised score for each hypothesis. **(b)** Normalise them to get $P(\text{Flu} \mid \text{fever, ache})$. **(c)** Cold began nine times more likely than Flu. By what factor did the evidence move the odds, and what is the final odds ratio? **(d)** State what conditional independence assumes here, and give one concrete reason it may be false.

Solution.

(a) Prior $\times$ likelihoods, for each hypothesis

$$\text{score}(\text{Flu}) = 0.10 \times 0.90 \times 0.80 = \mathbf{0.072}$$

$$\text{score}(\text{Cold}) = 0.90 \times 0.20 \times 0.10 = \mathbf{0.018}$$

These are not probabilities. They are $P(H) \prod_i P(E_i \mid H)$, which is $P(H, E_1, E_2)$, and they do not sum to 1 because the evidence itself has some probability.

(b) Normalise

$$\text{total} = 0.072 + 0.018 = 0.090$$

$$P(\text{Flu} \mid \text{fever, ache}) = \frac{0.072}{0.090} = \mathbf{0.8000}, \quad P(\text{Cold} \mid \cdot) = \frac{0.018}{0.090} = \mathbf{0.2000}$$

The two now sum to 1, as they must, since the system admits only these two possibilities.

(c) How far the evidence moved things

$$\text{prior odds (Flu : Cold)} = \frac{0.10}{0.90} = \frac{1}{9}$$

$$\text{combined likelihood ratio} = \frac{0.90}{0.20} \times \frac{0.80}{0.10} = 4.5 \times 8 = \mathbf{36}$$

$$\text{posterior odds} = \frac{1}{9} \times 36 = 4 : 1 \implies P = \frac{4}{5} = 0.8000 \quad \checkmark$$

Two symptoms turned 1:9 against into 4:1 in favour — a 36-fold swing, because independent evidence multiplies.

(d) *The independence assumption*

It assumes that *once you know the disease*, learning the patient has a fever tells you nothing further about whether they have body-ache:

$$P(\text{ache} \mid \text{Flu, fever}) = P(\text{ache} \mid \text{Flu})$$

That is what licenses multiplying 0.90 by 0.80.

It is very likely false here. Fever and body-ache share a mechanism — both follow from the inflammatory response — so among flu patients, the feverish ones are more likely than average to ache. The evidence therefore overlaps, and multiplying counts the same underlying signal twice.

The consequence is characteristic and worth recognising: naive Bayes tends to produce posteriors that are **too extreme** — 0.80 where the truth might be 0.65. Its *ranking* of hypotheses is usually still right, which is why the method survives despite an assumption that is almost never true.

F5 When a 99% test means almost nothing A screening test has sensitivity 0.99 and a false-positive rate of 0.05. The condition affects 1 person in 1000. (a) Compute $P(D \mid +)$. (b) Out of 100 000 people screened, how many test positive, and how many of those actually have the condition? (c) The sensitivity here (0.99) is higher than in the worked example (0.90), yet the posterior is far lower. Explain why, in one paragraph. (d) Which single number would you change to improve this test, and what would the posterior become if the false-positive rate fell to 0.005?

Solution.

(a) *The calculation*

$$\begin{aligned} P(D) &= 0.001, & P(\neg D) &= 0.999 \\ P(+) &= 0.99(0.001) + 0.05(0.999) = 0.00099 + 0.04995 = 0.05094 \\ P(D \mid +) &= \frac{0.00099}{0.05094} = \mathbf{0.0194} \quad (1.94\%) \end{aligned}$$

(b) *Out of 100 000 people*

	people	test +
have it	100	99
do not have it	99 900	4 995
total	100 000	5 094

Of 5 094 positive results, **99** are genuine and 4 995 are false alarms — 98.06% of everyone told they may be ill is not.

(c) *Why the better test does worse*

Because sensitivity is not the number that is failing. The likelihood ratio here is $0.99/0.05 = 19.8$, barely different from the worked example's 18; the test is doing roughly the same amount of evidential work in both cases.

What changed is the prior: 0.001 instead of 0.01, ten times smaller. Multiplying a ten-times-smaller number by a similar factor lands ten times lower, and 15.38% becomes 1.94%.

Said structurally: raising sensitivity from 0.90 to 0.99 can add at most 10% more true positives, since it only ever affects the hundred people who are ill. The false positives come from the ninety-nine thousand nine hundred who are not, and nothing about sensitivity touches them. When the well vastly outnumber the sick, the false-positive rate governs the answer and the sensitivity is almost irrelevant.

(d) *The number to change*

The **false-positive rate**, $P(+ | \neg D)$. Drop it from 0.05 to 0.005:

$$P(+) = 0.99(0.001) + 0.005(0.999) = 0.00099 + 0.004995 = 0.005985$$

$$P(D | +) = \frac{0.00099}{0.005985} = \mathbf{0.1654} \quad (16.54\%)$$

A tenfold improvement in specificity multiplies the posterior by roughly 8.5, from 1.94% to 16.54% — while the same effort spent on sensitivity could not have raised it past about 1.96%.

What it tells you

The engineering lesson is that the two error rates are not symmetric, and which one matters is decided by the base rate rather than by the test. For a rare condition, effort spent on sensitivity is nearly wasted and effort spent on specificity is decisive.

Note also that even 16.54% does not justify treating anyone. It justifies a second, different test — which is what every real screening programme actually does.

More practice Answers are given in brackets.

- i. $P(D) = 0.10$, $P(+ | D) = 0.80$, $P(+ | \neg D) = 0.10$. Find $P(+)$ and $P(D | +)$. [$P(+) = 0.08 + 0.09 = 0.17$; $P(D | +) = 0.08/0.17 = 0.4706$]
- ii. Same numbers, by natural frequencies out of 1000. Fill the table. [100 **ill**, 80 **positive**; 900 **well**, 90 **positive**; $80/170 = 0.4706$]
- iii. Give the likelihood ratio for that test, and the prior odds. [**LR** = 8; **prior odds** = 1/9; **posterior odds** = 8/9]
- iv. A test has $LR = 10$. At what base rate does one positive result make the condition exactly as likely as not? [**prior odds** = 1/10, **so** $P = 1/11 = 0.0909$]
- v. Two hypotheses have scores 0.024 and 0.036 after multiplying priors by likelihoods. Give both posteriors. [0.4 **and** 0.6]
- vi. Why does a very low base rate make a highly sensitive test nearly useless? [**the healthy majority produces far more false positives than the sick minority produces true ones**]

Syllabus map. Part A — uninformed search: BFS and DFS, completeness, optimality and complexity (A1, A2), why BFS is optimal only under equal step costs (A3), incompleteness and depth limits (A4). Part B — heuristic functions: the two 8-puzzle heuristics and dominance (B1), admissibility (B2), consistency (B3), effective branching factor (B4). Part C — informed

search: the evaluation function (C1), A traces (C2), Best First Search and why it is not optimal (C3), Hill Climbing with local optima, plateaus and restarts (C4), the cost of an overestimating heuristic (C5). Part D — knowledge representation: inheritance in semantic networks (D1, D3), frames with defaults and overrides (D2). Part E — production and expert systems: forward and backward chaining (E1), propositional logic and entailment by truth table (E2, E3), first-order predicate logic and quantifiers (E4), conflict resolution (E5). Part F — reasoning under uncertainty: Bayes' theorem (F1), sequential updating (F2), base rates and prevalence (F3, F5), several pieces of evidence combined (F4).*